Toward Effective LLM-Based Automated Program Repair: A Taxonomy of Prompting Strategies and Key Trends, Insights, and Challenges

Omar Messarhi[a], Mohamed Lamine Kerdoudi[a,b], Okba Tibermacine[c]

[a]*RLP Lab and Computer Science Department, University of Biskra, Algeria*
[b]*IRISA and Université Bretagne Sud, Vannes, France*
[c]*National School of Artificial Intelligence Algiers, Algeria*

Abstract

Automated Program Repair (APR) remains a core challenge in software engineering, primarily constrained by the rigid boundaries of search-based and template-driven methods. The emergence of Large Language Models (LLMs) has catalyzed a paradigm shift, enabling prompting-based approaches that exploit pre-trained reasoning capabilities to repair code without extensive fine-tuning. However, the rapid proliferation of these techniques lacks a systematic categorization framework, obscuring their unique functions and hindering practical application. In this paper, we present a taxonomy and analysis of recent prompting-based APR research. We propose a classification that organizes prompting strategies into three distinct families: Context and Structure Awareness, Cognitive Reasoning Enhancement, and Quality Assurance. Our findings highlight a highly concentrated research landscape; specifically, 66% of the reviewed studies rely on GPT-3.5 or GPT-4, and (54.7%) evaluate their approaches using the *Defects4J* benchmark. More critically, we identify systemic methodological limitations that limit real-world validity: 34% of studies rely on perfect fault localization, and 41.5% target single-function bugs, assumptions that significantly inflate performance metrics. This taxonomy synthesizes the state-of-the-art, exposes these "reality gaps" and outlines concrete research directions to transition prompting-based APR from academic benchmarks to industrial deployment. The replication package of this survey is available at: https://github.com/messarhi/prompt4APRtax/.

Keywords: Automated Program Repair, Large Language Models, Prompting Strategies, Software Debugging, Program Synthesis, LLM-based APR.

1. Introduction

Software bugs represent a persistent and costly challenge in modern software development. Studies indicate that debugging activities consume approximately 50% of developers' time [5]. Automatic Program Repair (APR) aims to automate the process of identifying and fixing software defects.

Traditionally, the field has been dominated by "generate-and-validate" (G&V) architectures. As established by Martinez and Monperrus in their exploration of the Astor framework [40], the design space of G&V repair is vast, encompassing various strategies for fault localization, ingredient selection, and transformation operators. However, these traditional approaches often rely on predefined search spaces or heuristic-based mutations, which can limit their flexibility when faced with complex, non-template-conforming bugs.

The emergence of large language models (LLMs) has shifted how APR techniques are designed and evaluated. Pre-trained on vast code corpora, models like GPT-4, Codex, and CodeLlama excel at understanding code semantics and generating syntactically correct patches, demonstrating strong performance across various software engineering tasks such as translation, documentation, and repair. This transition has expanded the applicability of APR to diverse domains, including specialized software architectures. For instance, Nguyen et al. [43] recently investigated automated repair for variability bugs in software product lines, demonstrating that fixing failing products individually and propagating patches is significantly more effective than attempting to repair all variants simultaneously.

Empirical evaluations such as those presented in [22] have consistently shown that LLM-based APR systems can achieve repair success rates that significantly exceed those of traditional approaches, representing measurable progress over traditional approaches. In fact, LLMs are primarily designed as general-purpose models to generate and process human-like text across a variety of contexts. These foundational models are trained on large corpora that combine natural language with source code, enabling them to comprehend the logic, syntax, and documentation for programming across different languages. However, their generic architecture, while offering broad applicability, poses challenges in highly specialized tasks such as program repair.

Despite the impressive zero-shot capabilities of LLMs in generating plausible code, their effectiveness in APR is highly sensitive to the way the repair task is presented. This sensitivity has given rise to the field of prompt engineering, where

Email addresses: omar.messarhi@univ-biskra.dz (Omar Messarhi), l.kerdoudi@univ-biskra.dz (Mohamed Lamine Kerdoudi), okba.tibermacine@ensia.edu.dz (Okba Tibermacine)

the structure, context, and instructions provided to the model become the primary drivers of repair performance. While initial efforts focused on simple "fill-in-the-blank" or instruction-following prompts, modern strategies have evolved into complex, multi-stage pipelines. These include iterative refinement loops, chain-of-thought reasoning, and the inclusion of diverse context such as error messages, execution traces, and similar code snippets.

Although individual studies [68, 76, 81, 4, 26, 28] investigate specific aspects of LLM-based APR, a systematic synthesis evaluating diverse prompting strategies and providing empirical insights for researchers and practitioners remains lacking. Existing surveys on APR largely focus on traditional techniques, with limited coverage of recent LLM-based approaches. Furthermore, the rapid evolution of LLM capabilities and prompting methodologies creates a moving target, necessitating up-to-date empirical analysis and theoretical understanding.

Several recent surveys have examined the growing role of LLMs in APR. Earlier reviews survey APR techniques by paradigm and tool performance [20, 11]; others study prompt engineering outside the repair domain [58, 53, 18]; a recent group targets LLM-based APR directly, organizing it by fine-tuning strategy [84], architectural integration [77], or paradigms of control authority [71]. Among these, the systematic review by Zhang et al. [78] offer broad coverage of the field, mapping LLM-based APR across adaptation paradigms, model architectures, and repair scenarios. This breadth is valuable for understanding the overall landscape, but it also means that prompting is treated as one adaptation strategy among several, alongside fine-tuning and agentic repair, rather than as an object of study in its own right. The result is a resolution mismatch: they report zero-shot prompting in 54 of 189 studies and few-shot in 24, but an instruction-only prompt, a role-conditioned prompt, and one supplying failing-test output are indistinguishable within that zero-shot count despite differing in repair effectiveness. Yang et al. [71] and Zhan et al. [77] are similarly coarse on this axis. Consequently, existing reviews do not disentangle prompting mechanisms with the granularity needed to determine which specific strategies are effective, under what conditions, and for which classes of defects.

This paper addresses that gap by introducing a systematic taxonomy focused exclusively on prompting-driven repair, in which the LLM's parameters remain frozen and the prompt itself constitutes the primary variable under study. Rather than treating prompting as one technique among several for adapting an LLM to the repair task, we treat it as a distinct methodological layer with its own design space, and we build our taxonomy and research questions around that design space specifically, rather than around the broader population of LLM-based APR studies. Fine-tuning and agentic architectures are accordingly treated as orthogonal levers and fall outside our scope.

Emphasizing recent LLM-driven methods over traditional APR, we analyze how advanced prompting techniques (e.g., Chain-of-Thought, Few-Shot prompting) influence repair performance across bug types, model architectures, and programming languages. We further examine how model characteristics such as scale, training data, and access modality interact with these strategies. Ultimately, we identify key challenges and open research directions, aiming to build a consolidated knowledge base that helps researchers and practitioners select the most effective prompt-based APR solutions for their specific contexts. Our main contributions are as follows:

- A taxonomy of LLM prompting strategies for APR, categorized by mechanism, input requirements, and bug type suitability, which distinguishes strategies that broader surveys have previously grouped together.
- An empirical synthesis of existing studies that highlights trends, performance trade-offs, and best practices for models, prompting, and evaluation protocols.
- A roadmap of open challenges, future research directions, and practical recommendations for prompt strategies, LLM selection, and evaluation practices across diverse APR scenarios focusing specifically on prompt-driven repair rather than general LLM-based APR.

The remainder of this paper is organized as follows: Section 2 details our methodology, and Section 3 presents our prompting taxonomy for LLM-based APR. Section 4 summarizes our empirical findings, while Section 5 outlines open challenges in the field. Next, Section 6 discusses practical implications, Section 7 addresses threats to validity, and Section 8 reviews related work. Finally, Section 9 concludes the paper and outlines directions for future work.

2. Methodology

Adapting Kitchenham and Charters' guidelines [27] and the PRISMA framework [47], we conduct a systematic review of LLM-based APR through four sequential phases: (1) research question formulation, (2) systematic search and selection, (3) five-dimensional data extraction, and (4) structured narrative synthesis.

2.1. Research Questions

The primary objective of this study is to build both a taxonomy of prompting strategies for LLM-based APR and an empirically grounded understanding of *when, why, and at what cost* specific strategies succeed or fail, enabling researchers to identify open problems and practitioners to select appropriate strategies for their repair scenarios. This goal stems from a critical gap in current literature: while prompting is central to LLM-based repair, existing surveys largely treat prompt design as an implementation detail rather than an independent object of study.

To guide literature selection, data extraction, and synthesis, this study addresses four cumulative Research Questions (RQs) that progress from structural classification (*what*), to empirical effect (*when/how well*), mechanistic explanation (*why*), and trade-off analysis (*at what cost*):

- **RQ1**: *How can the design space of prompting strategies in LLM-based APR be taxonomized ?*

Rationale. The literature currently lacks a shared vocabulary for prompting-based APR. Studies describe prompt designs using inconsistent terms and varying levels of granularity, making it impossible to compare effectiveness claims without fixed category boundaries. RQ1 addresses this gap by establishing a taxonomic foundation for the rest of this study. Without a unified framework to distinguish prompting patterns (e.g., retrieval augmented vs. template-guided prompts), any subsequent evaluation of repair performance or underlying mechanisms would rest on an unstable basis.

- **RQ2**: *To what extent, and under what conditions, do specific prompting strategies affect APR efficacy ?*

  Rationale. With prompting strategies taxonomized in RQ1, the next critical question is whether these structural distinctions matter empirically. Because the primary studies are methodologically too heterogeneous for a meta-analysis, RQ2 forms the empirical core of this study by synthesizing reported outcomes across the taxonomy. This evaluates whether prompt design directly drives repair success independently of model or dataset choices. In turn, identifying these performance gaps directly motivates RQ3, which investigates the underlying mechanisms driving observed efficacy differences.

- **RQ3**: *How do model characteristics such as architecture, scale, training data, and access modality affect the effectiveness of prompt-based repair?*

  Rationale. Attributing repair outcomes solely to prompt design risks confounding prompt efficacy with the capabilities of the underlying backbone model. A given strategy will not perform identically across models, as performance is heavily moderated by parameters like model scale, pre-training corpus, or whether the model is accessed via open weights or a closed API. RQ3 treats model characteristics as a distinct axis of analysis. This separation allows the study to isolate which repair outcomes are attributable to prompt construction versus backbone model capabilities, ensuring that the empirical conclusions in RQ2 are interpreted accurately rather than overstated.

- **RQ4**:*What are the primary limitations and open challenges in current prompt engineering for APR?*

  Rationale. Even after accounting for prompting strategies and model characteristics, current approaches exhibit persistent shortcomings—including high computational costs, extreme prompt sensitivity, context length limits, and poor generalization to domain-specific bugs. RQ4 completes the investigative sequence by analyzing these residual limitations. Building on the foundation of RQ1–RQ3, RQ4 synthesizes these challenges, which can only be meaningfully interpreted once the taxonomy, empirical trends, and model-level moderators are established, to ground the forward-looking research agenda presented in Section 5.

2.2. Search Strategy

While early pre-trained code models [15, 60] and subsequent code-capable LLMs [8] relied primarily on fine-tuning, the early 2022 emergence of diverse prompting techniques, including zero-shot, few-shot in-context learning, and CoT reasoning shifted APR toward prompt-driven paradigms. Accordingly, our search spans January 2022 to June 2026, coinciding with this shift and providing broad, up-to-date coverage of prompt engineering's rise as a primary variable in the field.

We combined automated database queries with manual backward and forward snowballing. To support open science and full reproducibility, our search protocol, filtering scripts, and replication package are publicly available on GitHub at `https://github.com/messarhi/prompt4APRtax/`.

2.2.1. Data sources & Search String Construction

To ensure comprehensive coverage of LLM-based APR literature, we followed established survey protocols [77, 78]. We first manually searched premier software engineering venues—six conferences (ICSE, FSE, ASE, ICSME, SANER, and ISSTA) and five journals (TSE, TOSEM, EMSE, JSS, and IST)—yielding an initial seed set of 21 relevant studies. Using this seed set, we developed and refined a search strategy structured around three keyword dimensions: (i) LLM-related terms, (ii) Prompt terms, (iii) APR-related terms. We constructed database-adapted Boolean queries by combining synonyms within each dimension using `OR` and joining the three dimensions with `AND`:

- **LLM terms**: *large language model*, *LLM*, *GPT*, *CodeLlama*, *StarCoder*, *Codex*, *transformer model*, *PLM*, *CodeT5*, *pretrained*, *pre-trained*

- **Prompt terms**: *prompt engineering*, *prompting*, *few-shot*, *zero-shot*, *chain-of-thought*, *CoT*, *ToT*, *React*, *RAG*, instruction tuning.

- **Repair terms**: *program repair*, *automatic repair*, *bug fix*, *patch generation*, *code repair*, *defect repair*, *APR*, *automated program repair*, *bug repair*

The canonical search strategy is formally expressed as the intersection of these three term sets:

$$\text{Query} = ((\text{LLM terms}) \vee (\text{Prompt terms})) \wedge (\text{Repair terms})$$

Finally, we executed these queries across the ACM Digital Library, IEEE Xplore, Scopus, and Google Scholar to retrieve peer-reviewed articles and preprints. Because each search interface enforces unique syntax constraints, the canonical query was adapted for each database (see Table A.7 for the exact strings). For instance, the query instantiated for Scopus targeting titles, abstracts, and keywords is structured as:

> (**TITLE-ABS-KEY**("large language model" **OR** LLM **OR** GPT **OR** CodeLlama **OR** StarCoder **OR** Codex **OR** "transformer model" **OR** PLM **OR** CodeT5 **OR** pretrained **OR** "pre-trained")

OR
TITLE-ABS-KEY("prompt engineering" **OR** prompting **OR** "few-shot" **OR** "zero-shot" **OR** "chain-of-thought" **OR** CoT **OR** ToT **OR** ReAct **OR** RAG **OR** "instruction tuning"))
AND
TITLE-ABS-KEY("program repair" **OR** "automatic repair" **OR** "bug fix" **OR** "patch generation" **OR** "code repair" **OR** "defect repair" **OR** APR **OR** "automated program repair" **OR** "bug repair")

2.2.2. Snowballing search

Following Wohlin's guidelines [63], the initial pool of studies retrieved via automated search was supplemented through backward and forward snowballing. Specifically, we manually examined the reference lists (backward snowballing) and citations (forward snowballing) of influential foundational papers from premier software engineering (ICSE, FSE, TSE) and machine learning (NeurIPS, ICLR) venues. This process identified 11 additional candidate papers, which were subsequently evaluated against the same study selection criteria.

2.3. Selection Process

We adapted the PRISMA guidelines [47] to establish a transparent and replicable study selection process. At every stage, two authors independently screened records, with a third author resolving discrepancies through consensus. The selection process proceeded through four stages:

1. **Identification:** Retrieving candidate literature through automated Boolean search queries across target databases, supplemented by citation snowballing.
2. **Deduplication:** Removing duplicate entries across databases as well as preprint/published version pairs.
3. **Title and Abstract Screening:** Filtering unique records against inclusion and exclusion criteria (Section 2.3.1) to remove out-of-scope work.
4. **Full-Text Evaluation:** Conducting an in-depth reading of candidate papers to confirm compliance with selection criteria.

2.3.1. Selection Criteria

We defined explicit selection criteria to isolate empirical contributions to prompt-driven automated program repair (APR) and filter out tangentially related work. Each criterion is structured to ensure transparent and traceable screening decisions.

- **Inclusion criteria:**
  1. **Relevance:** The study is a peer-reviewed paper or substantial preprint that explicitly focuses on LLM-based program repair, where prompt design drives patch synthesis.
  2. **Rigor & Evaluation:** The study presents quantitative empirical evaluations on established benchmarks (Defects4J, QuixBugs, HumanEval-Fix, SWE-bench, or ManyBugs).
  3. **Reproducibility:** The study reports the specific elements needed to replicate the prompting procedure under study, namely the prompt template, the exact model name and version, the source and number of few-shot exemplars (when used), as well as the decoding parameters (temperature and, where applicable, top-p). A study missing more than one of these elements was excluded on reproducibility grounds.
- **Exclusion criteria:**
  1. **Non-LLM technique:** The repair technique is search-based, constraint-based, template-based, or uses a non-LLM neural model.
  2. **Auxiliary task:** The LLM performs secondary tasks (fault localization, test generation, patch validation), while patch synthesis itself is non-LLM or non-prompt-driven.
  3. **Security-only focus:** Addresses exclusively security vulnerabilities (CVE/CWE patching, exploit mitigation) rather than general functional bugs (studies covering both are retained).
  4. **Non-empirical:** The study does not report original empirical results on prompting (position papers, opinion articles, unevaluated tool demonstrations, or surveys/SLRs).
  5. **Opaque reporting:** Fails to disclose prompt templates, construction procedures, or exemplar selection details required for replication.

2.3.2. Search and Selection Results

As shown in Table 1, the automated database search yielded 2002 records prior to filtering. An additional 11 candidate studies were identified through snowballing, bringing the total to 2013 papers retrieved before applying inclusion and exclusion criteria.

Table 1: Search results

Database	Records Retrieved
Google Scholar	1089
Scopus	544
IEEE Xplore	178
ACM DL	191
Snowballing	11
Total	2013

Figure 1 illustrates the PRISMA flow diagram for our study selection process, detailing the number of papers retained and excluded at each screening stage.

As shown in Figure 1, after duplicate removal, 1,227 unique records underwent title and abstract screening. Two reviewers independently screened all records, demonstrating almost perfect agreement (Cohen's $\kappa = 0.81$). This stage retained 307 records for full-text evaluation. The same reviewers independently assessed these full texts against the eligibility criteria,

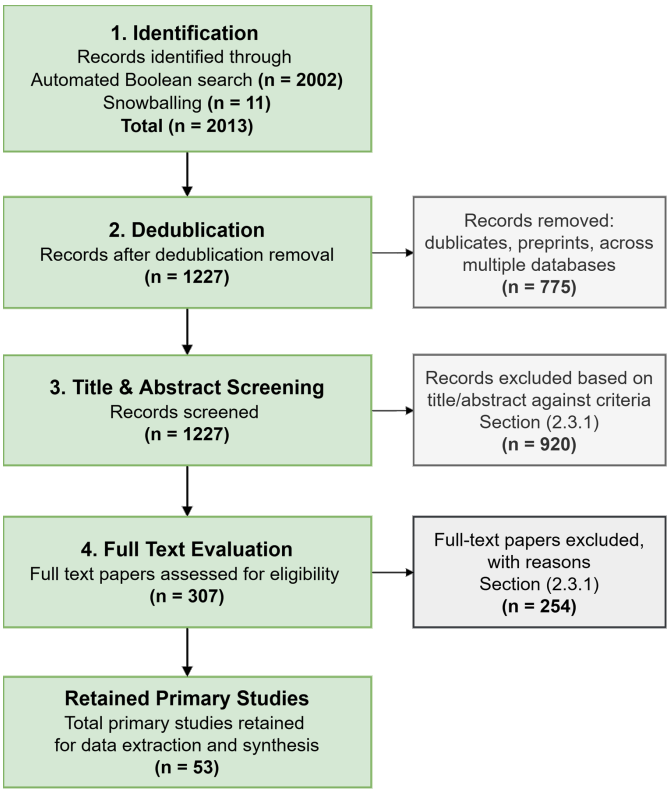


Figure 1: PRISMA flow diagram detailing paper counts across all stages of the selection process.

again achieving high inter-rater reliability (Cohen's $\kappa = 0.85$). Following the exclusion of 254 records, 53 studies were selected for the final corpus.

2.4. Data Extraction Dimensions

We synthesized the LLM-based APR landscape using a five-dimensional extraction framework:

- **Prompting techniques:** we categorized each study by its dominant prompting strategy to develop a systematized taxonomy representing the primary mechanisms used to guide, process, and refine the LLM's generation.
- **LLM characteristics:** we extracted model family (e.g., GPT, CodeLlama) to track architectural trends, model size to correlate scale with performance, and access type (proprietary vs. open-source) to assess transparency.
- **Evaluation metrics:** we classified metrics into correctness (e.g., exact matches, test pass rates), intermediate quality (e.g., compilation success, syntactic validity), and efficiency (e.g., time per repair, token usage).
- **Benchmarks and datasets:** we cataloged key dataset attributes (defect volume, language diversity, typology, and complexity), highlighting studies that use cross-benchmark evaluations to demonstrate robustness and transferability.
- **Reproducibility artifacts:** we documented the public availability of prompts, repair code, and evaluation data to assess study quality and facilitate future replication.

3. Prompting Taxonomy for LLM-based APR

To systematize the landscape of prompting-based APR, we classify the existing prompting methods into three high-level categories (see Figure 2).

1. **Structure and context-based strategies**: Focus on enriching the input by providing syntactic structure and code context to guide the LLM's generation.
2. **Cognitive and reasoning enhancement**: Aim to improve logical processing through mechanisms such as stepwise reasoning or Chain-of-Thought (CoT) prompting.
3. **Quality assurance and reliability**: Prioritize correctness through post-generation validation, test execution, or iterative refinement loops.

The boundaries of these three categories are anchored directly in the sequential operational phases of a standard LLM generation pipeline: input formulation, reasoning generation, and output validation.

- **Input Formulation & Contextual Prompting**: Enriching prompt structure and retrieving repository context to guide the initial generation phase.
- **Reasoning & Cognitive Enhancement**: Elevating logical processing during generation via mechanisms like Chain-of-Thought (CoT) or stepwise reasoning.
- **Output Validation & Quality Assurance**: Ensuring post-generation correctness through test execution, static analysis feedback, or iterative refinement loops.

Because hybrid strategies can span multiple categories (e.g., Retrieval-Augmented Generation influencing both context and reasoning), we classify each study by its *dominant trait*—defined as the primary experimental variable isolated and evaluated in the original work. For instance, if a paper introduces RAG primarily to assess its impact on multi-step logic, it is coded under *Reasoning & Cognitive Enhancement*. Conversely, if RAG is evaluated strictly on retrieving missing repository context without altering the reasoning process, it falls under *Input Formulation & Contextual Prompting*.

To validate this taxonomy, two authors independently coded the complete corpus following these explicit guidelines. Inter-coder reliability yielded a Cohen's Kappa of $\kappa = 0.85$, indicating strong agreement. Initial discrepancies were resolved through consensus discussions.

Finally, Table 2 serves as a cross-reference table mapping each reviewed paper to its primary taxonomy category and subcategory, detailing per-subcategory paper counts and complete citation lists. Additionally, while *role prompting* frequently serves as an embedded prompt component across several studies, it is never a primary strategy, which is why no papers are mapped directly to it.

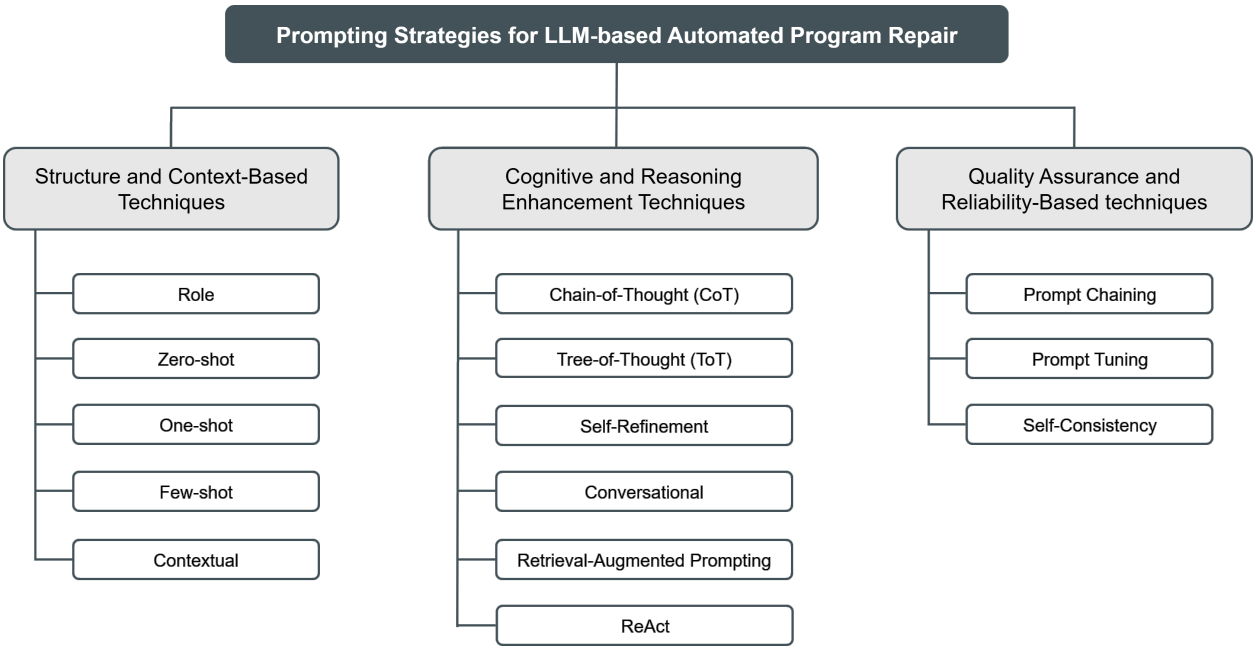


Figure 2: Taxonomy of prompting strategies in LLM-based APR.

3.1. Structure and Context-based Techniques

This category establishes the core elements of prompt design: construction (the 'how'), provided context (the 'what'), and model persona (the 'who').

3.1.1. Role Prompting

Role prompting is a technique that involves guiding the LLM behavior by specifying a specific role (or persona) for it which is believed to influence its responses. Instead of getting generic responses, the LLM is asked to "role-play" as a particular type of person or expert [54]. The handling of role prompting has undergone a distinct evolution from earlier LLMs to the newest ones. By the time [61] introduced Instruction Tuning, and later [45] introduced Reinforcement Learning (RLHF), models like GPT-4 can interpret the intent behind a prompt without needing examples. Instead of treating the words "You are an expert debugger" as more text to complete, the model treats it as an instruction to shape its entire output. The increased sophistication of newer models has somewhat reduced the standalone effectiveness of this approach. ExpertPrompting [69] refines role prompting by using In-Context Learning to automatically generate a detailed, domain-specific expert identity (e.g., physicist, engineer) tailored to each instruction. Multi-Expert Prompting [37] extends ExpertPrompting by generating multiple distinct expert identities simultaneously. Each expert independently responds to the query, and outputs are aggregated to select the best response.

In APR, role prompting involves instructing the LLM to take on the role of an experienced software developer, debugger, or domain expert to improve the quality of bug fixes.

3.1.2. Zero-shot Prompting

Zero-shot prompts are completely based on natural language directives that make use of the model's pre-trained knowledge. Usually, it consists of a task and a role (optional). The model processes a given directive without being provided any examples and then generates a patch. This technique tests the ability of the model to generalize from its training data to new tasks without additional context or guidance.

Xia et al. [67] introduced AlphaRepair, the first cloze-style APR approach by using CodeBERT, a large pre-trained code model in a zero-shot setup without fine-tuning. Instead of translating buggy code to fixed code, AlphaRepair masks the buggy line and lets the model iteratively fill tokens, then re-rank candidate patches by computing likelihood scores and manually inspect their correctness against tests.

The study of Prenner et al. [49] involved using OpenAI's Codex as a zero-shot program repair tool. They designed five prompts that differed in their input content to interact with Codex. Using a benchmark of 40 common coding errors in Python and Java, Codex fixed 23 Python bugs and 14 Java bugs under the best configuration, making it competitive with SOTA neural APR tools despite no task-specific training.

In the study of Fan et al. [14], for each of the 113 tasks on their LMDefects dataset, Codex was asked to generate 50 candidate solutions through a zero-shot prompt consisting only of a problem description; no input or output examples were included. This resulted in 46 solved and 67 unsolved tasks.

Wuisang et al. [64] examined the out-of-the-box effectiveness of GPT-3.5 for APR using a single, fixed zero-shot prompt. They evaluated the model on 40 Python programs from the QuixBugs benchmark, submitting each buggy program four times to capture output variability. GPT-3.5 successfully repaired 25 of the 40 bugs, surpassing both Codex and a conventional APR

Table 2: Mapping of 53 reviewed studies across taxonomy categories and prompting strategies

Category	Prompting Strategy	Study Count/ 53	Studies
Structure and context-based	Zero-Shot	6	Prenner et al. (2022) [49], AlphaRepair (2022) [67], Fan et al. (2023) [14], Wuisang et al (2023) [64], Omari et al. (2024) [44], Hu et al. (2025) [19]
	Few-Shot	3	Xia et al. (2023) [66], Cedar (2023) [42], Do et al. (2023) [12]
	One-Shot	1	D4C (2025) [70]
	Contextual	6	LDB (2024) [83], Qu et al. (2024) [50], Haque et al. (2025) [17], Cao et al. (2025) [7], Liu et al. (2026) [34], HAFix (2026) [55]
Cognitive and reasoning enhancement	CoT	7	ThinkRepair (2024) [76], Maniple (2025) [48], AutoSD (2025) [26], CodeCorrector (2025) [32], Darwiyanto et al. (2025) [10], Ehsani et al. (2025) [13], TypeRepair (2026) [79],
	ToT	1	CodeFixer (2024) [33]
	Self-Refinement	2	DRCodePilot (2024) [82], REx (2024) [57]
	Conversational	4	Sobania et al. (2023) [56], ChatRepair (2024) [68], CREF (2024) [72], ContrastRepair (2025) [28]
	Retrieval-Augmented Prompting	9	InferFix (2023) [24], RING (2023) [25], RLCE (2024) [9], RAGFix (2024) [39], RetypeR (2024) [16], DSrepair (2025) [46], ReAPR (2025) [36], DeepK (2026) [31], ExpeRepair (2026) [41]
	ReAct	2	Passerine (2025) [51], RepairAgent (2025) [4]
Quality assurance and reliability	Prompt Chaining	10	AutoCodeRover (2024) [81], RGD (2024) [23], Robin (2024) [3], PATCH (2025) [80], AdverIntent-Agent (2025) [75], SpecRover (2025) [52], UniDebugger(2025) [29], T^3 (2025) [35], Agentless (2025) [65], Abdinabiev et al. (2026) [1]
	Prompt Tuning	1	Cai and Jiang (2025) [6]
	Self-Consistency	1	Ahmed et al. (2023) [2]

system. When concise natural-language bug descriptions were added to the prompt, the model resolved five additional defects.

Omari et al. [44] measured ChatGPT's ability to detect and fix single-line Python bugs under three variations across 220 bugs. Under zero-shot prompting, GPT-3.5-turbo was able to detect and repair between 47.5% and 67.8% of the bugs. However, its performance drops significantly on obfuscated code (25%), it has a high false positive rate, and its fixes often require manual verification. The study concludes that ChatGPT is not yet a fully reliable solution for all code repair scenarios.

Hu et al. [19] conducted an empirical evaluation of GPT-family LLMs in APR, specifically the GPT-o1 series. They designed a two-step process to interact with the LLM, first the LLM is asked for a fix through a basic prompt consisting only of the instruction and the target buggy program. For any bug that was not fixed in the first step, the model is given a new prompt with error feedback for a second attempt. The GPT-o1 models successfully repaired all 40 bugs in *QuixBugs* benchmark.

Zero-shot prompting effectively resolves single-hunk errors but struggles with multi-hunk patches or insufficient context. Without explicit constraints, LLMs frequently rewrite entire code blocks instead of providing targeted fixes, hindering developers' understanding of the underlying defect. The evidence does not support a general claim that zero-shot prompting is "insufficient" for APR; rather, its adequacy depends on both model scale and the complexity of the benchmark: simpler algorithmic bugs (QuixBugs) are more effectively resolved via zero-shot repair than real-world project bugs (Defects4J).

3.1.3. Few-shot Prompting

Few-shot prompting occupies a middle ground between zero-shot inference and full fine-tuning by supplying a limited set of input-output exemplars sufficient to guide the model toward the target task. Since LLMs lack native specialization in APR, this approach enables flexible adaptation across different codebases and bug categories.

Xia el al. [66] examined the feasibility of applying LLMs directly to APR. They employed few-shot prompting for complete function generation, where the model is prompted to generate entire patched functions. The prompt starts with a task description followed by two demonstration (bug-fix) pairs before the target buggy function. The few-shot approach's benefits in this work were clear; it eliminated the need for supervised bug-fix datasets or model fine-tuning, and facilitated performing fault localization and repair in a single step.

CEDAR [42] builds effective prompts for few-shot learning by using either sparse or dense retrieval methods to retrieve rel-

evant code demonstrations from a large pool of examples based on similarity to the target task. Cedar correctly fixed 51.72% of JavaScript bugs on the TFIX dataset with far fewer examples than fine-tuned models. This study shows that careful retrieval of few relevant examples can outperform larger random example sets and improve LLM effectiveness on APR tasks while avoiding costly task-specific training.

Do et al. [12] aim to automate line-level bug localization and program repair for student-written code without symbolic engineering or retraining. They use an LLM to generate in-context few-shot examples from each buggy program by summarizing the bug, generating a correct implementation, and mutating it to produce paired demonstrations used for localization and repair prompts. Evaluated on the QuixBugs Python benchmark, the method improves stability and overall performance over zero-shot and naïve few-shot baselines.

These strategies demonstrate that few-shot prompting can improve repair accuracy across diverse scenarios, mitigating reliance on large labeled datasets and task-specific fine-tuning. However, these gains often incur increased generation latency and occasional degradation in compilation success rates relative to alternative methodologies. Several limitations persist in few-shot APR: context length constraints that cap the number and complexity of demonstrations. Additionally, models struggle with intricate semantic bugs, exhibit tendencies to overfit to surface-level patterns, and face scalability challenges in maintaining high-quality, diverse example repositories.

3.1.4. One-shot Prompting

One-shot prompting bridges zero- and few-shot methods by demonstrating the desired repair pattern with a single bug-fix example. D4C [70] leverages the next-token prediction objective of decoder-only LLMs, enabling GPT-4 to perform implicit fault localization and repair simultaneously. D4C synthesizes a concise bug report from contextual artifacts, including documentation, failed tests, and error messages. The model is then prompted with a one-shot example to generate a refined function. Sampling 10 patches per bug, it fixed 180 of 437 *Defects4J* bugs.

While introducing a single example reduced both generation perplexity and malformed outputs, these benefits rely heavily on exemplar quality, a poor example can actively mislead the model. Furthermore, the approach's effectiveness remains uncertain when high-quality contextual artifacts are unavailable.

3.1.5. Contextual Prompting

Advances in models like GPT-4o [21] have improved context comprehension and response precision, enabling more sophisticated prompt engineering approaches. Contextual prompting moves beyond generic queries by injecting task-relevant cues (such as conversation history, details specific to the task, or user preferences) directly into the prompt. In APR, this means supplying the LLM with full bug context so that patch generation is grounded in accurate diagnostic information.

Zhong et al. [83] introduced LDB, a framework to incorporate runtime execution traces into LLM-based debugging. It breaks each program into basic blocks via control flow graphs and then asks the LLM to verify them step-by-step against the task. The LLM is iteratively prompted with debugging feedback until all visible tests pass or max iterations (10) are reached. LDB consistently boosted the performance of all LLM backbones across all benchmarks.

Qu et al. [50] examined how different forms of context affect ChatGPT's ability to fix logic bugs. Using the FIXEVAL dataset, they demonstrate that natural-language problem descriptions yield significantly higher repair accuracy than test cases alone. Performance peaked at 83.3% when these descriptions have been combined with both passing and failing tests.

Haque et al. [17] investigated the effectiveness of execution traces by dynamically reducing their complexity. For each buggy program and its failing test case, they use `PySnooper` to generate an execution trace. Then, they designed a series of progressively augmented prompts that include or omit the traces. Across all three datasets, the optimized prompt yielded the most consistent performance improvements.

Cao et al. [7] evaluated the capabilities of ChatGPT in debugging and repairing 58 buggy deep learning (DL) programs. The study designed an enhanced prompt template that included explicit context (task, dataset, and repair objective). The authors submitted each prompt five separate times, gpt-3.5-turbo solved 43 of the 72 faults better than basic prompt.

Liu et al. [34] investigated how buggy code information influences the repair performance of LLMs. The evaluation on the QuixBugs Python subset shows that descriptive clues and exact fault locations increase repair rates across all model sizes, whereas test file content frequently acts as noise that degrades outcomes. The observed benefits of combined information are conditional on model capability, with commercial-scale models leveraging full-information inputs to achieve higher pass rates while smaller models suffer from context overload. These results empirically reinforce the distinction between universally useful information and capability-dependent information within the broader taxonomy of input augmentation strategies for APR.

HAFix [55] augments LLM inputs with blame-commits metadata. By evaluating seven history heuristics across 167 single-line Java and Python defects, HAFix-Agg increases Pass@k bug-fixing rates by over 45% on average against history-agnostic baselines. Furthermore, instruction prompts outperform masking, and early-stopping cuts inference costs by roughly 70% with limited performance loss. Yet, perfect localization and single-line constraints limit complex defect applicability.

These approaches show that grounding LLMs in relevant context and runtime states improves accuracy and reduces hallucinations. Conversely, bloated or irrelevant prompts disrupt the model's focus, lower output quality, and needlessly increase token costs. Table B.8 in Appendix B provides the complete per-study breakdown, including prompt formulations, evaluation setups, reported results, and underlying assumptions.

3.2. Cognitive and Reasoning Enhancement Techniques

This category includes techniques that enhance the model's reasoning by prompting explicit, step-by-step thought processes. Rather than simply providing a final answer, these prompts in-

struct the model to show its work, weigh multiple perspectives, or deconstruct complex problems into manageable parts.

3.2.1. Chain-of-Thought Prompting

The Chain-of-Thought prompting (CoT) [62] mimics human problem-solving by requiring the model to "think aloud" by generating causal explanations and intermediate steps before of producing its final response. In APR, CoT mirrors a developer's systematic debugging workflow: identifying code intent, diagnosing the root cause, determining a fix strategy, and finally composing the patch to achieve more precise results.

ThinkRepair [76] uses structured CoT prompts, including a role designation, task description, buggy function, and CoT indicator, to guide step-by-step bug repair, generating a high-quality knowledge base and proving CoT effectively improves LLM reasoning in program repair.

Parasaram et al. [48] investigated trace augmentation by evaluating which bug-related "facts" to include in prompts. They propose Maniple, a trained model that dynamically selects the optimal combination of seven diverse "facts" based on specific bug features. Enhanced by a CoT directive, Maniple fixed 88 out of 157 test bugs, achieving a 17% improvement over the best baseline.

AutoSD [26] employs an initial prompt with labeled headers to generate hypotheses about the bug's cause, and then integrates tool-grounding so the model produces executable debugger commands whose outputs are fed back into the prompt. This iterative CoT approach allows ChatGPT to build upon prior steps to reach a conclusion, while a special token `<DONE>` signals when debugging is complete. Surpassing LLM-only baselines, AutoSD correctly fixed 76 Java bugs on Defects4J v1.2, 113 Java bugs on Defects4J v2.0, and 187 Python bugs on ARHE.

CodeCorrector [32] mimics a developer's reasoning through a multi-stage LLM pipeline. It first distills a high-level repair goal from a failing test, uses that goal to retrieve relevant project-wide context, and finally combines the goal with local and global contexts to generate, compile, and validate patches.

Darwiyanto et al. [10] demonstrate that few-shot CoT prompting generally improves LLM bug-fixing accuracy on single-function defects. Using a single buggy-to-fixed example and a five-step reasoning instruction, DeepSeek-V3 achieved the best performance (36.6 plausible patches) at the lowest cost ($0.006 per run). However, the benefits of CoT are not universal, as models like o1-mini exhibited degraded performance under this prompt structure.

TypeRepair [79] exemplifies a modular, multi-stage CoT prompting strategy that forces the model to perform intermediate reasoning steps across the multi-stage pipeline, requiring it to state its analysis of the fault type and fix direction before producing code. Empirical validation on Defects4J and QuixBugs confirms that separating defect types drives significant correctness gains while protecting against context bloating. However, the approach's effectiveness depends critically on the reliability of the model's type classification, and the proposed ten-category taxonomy.

Prior studies highlights that step-by-step reasoning helps LLMs identify fix patterns, filter irrelevant signals, and leverage internal knowledge, ultimately improving repair accuracy relative to unstructured zero-shot baselines, with reported gains ranging from +6% to +377%. However, these gains introduce notable trade-offs, including higher computational costs, an increased risk of hallucinated reasoning traces, and a strict reliance on large-scale models. Importantly, CoT prompting is largely ineffective for smaller models. Because they lack the latent reasoning capacity, semantic abstraction, and pre-training breadth necessary for multi-step repair, forcing them to articulate explicit steps fails to induce deeper reasoning and instead exacerbates their underlying limitations. Furthermore, CoT prompting can also degrade performance in models with built-in reasoning mechanisms and when prompts are overloaded with facts.

3.2.2. Tree-of-Thought

Tree of Thoughts (ToT) [73] extends CoT by exploring multiple reasoning paths within a tree structure. Unlike the token-level greedy decoding of CoT, ToT treats coherent "thoughts" as nodes, using a search policy to enable self-evaluation, look-ahead, and backtracking. Successful implementation hinges on robust evaluation criteria, balancing exploration breadth against computational overhead, and scaling search depth to task complexity.

Recent research, notably the CodeFixer framework [33] integrates LLM reasoning with tree search for automated program repair. To optimize the process, CodeFixer filters irrelevant context, retaining only high-value information before prompting the engine. At its core, a fine-tuned GPT-3.5 Turbo serves as a patching engine within a ToT search, enabling high-confidence fixes through structured exploration. Evaluated on Defects4J and QuixBugs, CodeFixer significantly outperforms baselines, successfully repairing 92 and 40 bugs, respectively.

While this method mitigates common issues like context noise and invalid patches, it introduces significant practical challenges. The utility of intermediate reasoning paths is difficult to quantify, as test cases provide a surface-level signal that often fails to ensure semantic correctness. Furthermore, branching exploration significantly increases inference costs relative to linear prompting, while parallel repair trajectories can introduce ambiguity, ultimately complicating final patch selection.

3.2.3. Self-Refinement

LLMs often fail to generate correct solutions on their initial attempt. To address this, Self-refine [38] transforms the model into an autonomous critic and editor. This iterative, three-step feedback loop involves generating an initial response, critiquing the output, and refining the results until a specified quality threshold is achieved.

DRCodePilot [82] improves GPT-4 Turbo's full-match rate on 938 Jira issues by injecting Design Rationales (DR) mined from discussions into prompts. The LLM generates an initial patch using these DRs and refines it through a self-reflective feedback loop. Although DRCodePilot achieved 4.7× and 3.6× higher full-match rates than GPT-4 on Flink and Solr, absolute ratios remained low at 15.27% and 8.03%. These results underscore the significant challenges of repairing real-world industrial bugs.

REx [57] models iterative code refinement as a multi-armed bandit problem, treating each candidate program as an arm. At each step, REx uses Thompson Sampling to navigate the refinement space, prompting GPT-4 with the current program and a failing test case to generate improvements. By balancing exploration and exploitation, REx solved 28 of 38 tasks on the NLA benchmark while requiring 1.5× to 5× fewer LLM calls than standard baselines.

3.2.4. Conversational Prompting

Sobania et al. [56] empirically evaluate ChatGPT's ability to fix bugs in the 40 Python programs of the QuixBugs benchmark. Without any specialised training, ChatGPT fixes 19 bugs, far outperforming traditional APR approaches. Its performance is highly variable across runs, but by leveraging the dialogue system to provide a single hint, the success rate jumps to 31 out of 40 (77.5%). While the study evaluates the model's vanilla zero-shot capacity natively, it represents an early foundational work highlighting the potential of human-in-the-loop conversational AI to correct initial reasoning errors.

ChatRepair [68] utilizes automated, short conversational loops with validation feedback to improve LLM-based repair. By iteratively feeding incorrect patches and test failures back to the model, it enables the LLM to learn from previous attempts. This approach successfully fixed 162 bugs in the Defects4J dataset at an average cost of only $0.42 per bug.

ContrastRepair [28] pairs failing tests with highly similar passing tests to offer more precise, comparative feedback, enabling the LLM to better isolate root causes. It augments the prompt with these pairs, traceback, and dependent functions. The model then iteratively queries ChatGPT by either refining the previous patch or restarting from the original until it finds a plausible solution or exhausts the budget.

By iteratively learning from previous attempts and successes, these methods use conversational interactions to improve patch precision and reduce sampling costs. Weaker models benefit disproportionately from conversational/iterative prompting, suggesting that iterative refinement compensates for limited initial reasoning capability. However, extended dialogues introduce challenges such as catastrophic forgetting, accumulating misunderstandings across turns, and increased token consumption as the context window grows.

These observations indicate that conversational loops, while beneficial on average, can degrade if the model loses track of the original context or overfits to the most recent feedback.

3.2.5. Retrieval-Augmented Prompting

Retrieval-Augmented Prompting enhances standard prompts by inserting an explicit retrieval step to ground the model in task-relevant external context, such as repositories, documentation, or historical patches, rather than relying solely on parametric knowledge. The repair pipeline formulates a query from the initial input, fetches relevant artifacts from a knowledge base, and constructs an augmented prompt combining both sources.

Jin et al. [24] designed InferFix, an end-to-end solution that pairs a static analyzer with a retrieval-augmented LLM pipeline. It first uses the static analyzer to identify and localize bugs, then retrieves semantically similar historical bug–fix examples, and finally finetunes a 12B-parameter Codex model on prompts augmented with extended context and retrieved hints to generate fixes. On InferredBugs, InferFix reported high top-1 exact-match repair rates (76.8%) for Java and (65.6%) for C#.

RING [25] is a unified, prompt-based APR framework achieving cross-language generalization through similarity-based retrieval of repair-relevant examples from an external example bank, keyed on observed errors. It outperforms language-specific tools on Excel, Python, and C, but underperforms on PowerShell due to sparse PowerShell representation in Codex's pretraining corpus. Despite its multilingual design, performance remains critically bottlenecked by example bank coverage and quality.

RLCE [9] addresses complex repository-level APR by using static analysis to partition codebases into structured segments. Rather than processing the entire codebase, it retrieves only task-relevant segments and merges them into prompt templates for the LLM. On the RepoBugs benchmark, RLCE improved repair rates by up to 160% across three backbone LLMs compared to function-only baselines, confirming the necessity of repository-wide context.

RAGFix [39] improves accuracy by integrating RAG with a dynamic Stack Overflow knowledge base. The pipeline prompts the LLM to generate a high-level algorithm description from buggy code, embeds it as a vector, and performs cosine similarity search; if no close match is found, the system automatically retrieves new relevant posts to expand the knowledge base. The top-ranked answer is then passed into a second prompt directing the LLM to fix the original code. On 164 Python bugs, RAGFix raised Llama3-8B accuracy from 41.4% to 51.2% and Llama3-70B from 72.5% to 78.0%.

RetypeR [16] couples an AST-based with a fix ingredient retriever that operates on two levels. For every buggy program, it retrieves fix patterns from extern GitHub commits, combines them with the top-ranked donor identifier, and incorporates this knowledge into the repair prompt alongside an `<INFILL>` token to guide CodeT5 in code completion. The retrieved context allowed RetypeR to correct 62 and 29 Python errors on the TypeBugs and BugsInPy benchmarks, respectively.

DSrepair [46] conducted a large empirical study on the DS-1000 benchmark. It pairs a retrieved API knowledge from a knowledge-graph (DS-KG) based RAG with AST-based bug enrichment. The repair prompt consisted of the incorrect code snippet combined with those pieces of knowledge to guide the LLM toward producing fixes more effectively. The system improved the bug-fixing rate by 44.4%, 14.2%, 20.6%, and 32.1% respectively.

RAG-based repair in ReAPR [36] grounds patch generation in curated real-world bug-fix examples, comparing BM25 and DPR retrieval strategies. The top-ranked bug–fix pair is injected into a structured prompt alongside a target-project example to guide the LLM during function-level repair. With CodeLlama-7B and DPR, ReAPR fixes 68 bugs on Defects4J v2.0. However, the lower performance on GitBug-Java suggests that the effectiveness of RAG-based repair is strongly dependent on the quality and relevance of the retrieval corpus.

DeepK [31] constructs a structured knowledge base of root-cause diagnoses and repair strategies from historical bug-fix pairs, using patch execution to validate each extracted entry before indexing. Validation requires the LLM to generate a patch from its own knowledge; only entries producing correct patches are retained. For new buggy functions, embedding along task description, `AST`, and execution trace retrieves top-k knowledge entries, which are injected into the reasoning stage of an APR pipeline. Experiments on ACPR and AtCoder demonstrate consistent gains over LLM-based baselines, especially with stronger backbones like DeepSeek-v3.

ExpeRepair [41] combines episodic memory retrieval of past repair demonstrations with semantic memory recall to dynamically synthesize context-aware prompts for test and patch generation in a ReAct loop. By joining concrete few-shot demonstrations with abstract natural language rules, the combined framework increased repair rates to 60.3% pass@1 on SWE-Bench Lite and 74.6% on SWE-Bench Verified, outperforming standalone agentic and procedural baselines. The approach effectively addresses the limitations of static prompting. However, the strategy exhibited clear limitations in cold-start scenarios where no prior repair history existed, faced memory capacity bounds capped at fifteen entries per role, and remained constrained by initial bug reproduction accuracy.

These retrieval mechanisms underscore the value of API knowledge, verified Stack Overflow Q&A, and structurally relevant context for LLM-based APR, particularly for smaller LLMs with limited parametric knowledge, across both real-world and multi-file settings. Yet RAG introduces non-trivial practical costs: retrieval overhead imposes a measured 51% slowdown over plain-text retrieval, excessive API detail inflates token costs, and retrieved content may exceed context window limits. Furthermore, the effectiveness of retrieval-augmented prompting depends critically on retrieval corpus quality, irrelevant retrieval may mislead the model. These constraints collectively demonstrate that prompt relevance has a greater impact than prompt length: precise and relevant context matters more than sheer information density.

3.2.6. ReAct Prompting

ReAct [74] addresses the limitations of CoT by interleaving reasoning with action. The model generates internal reasoning traces to decompose problems, then executes tool or environment interactions, receives observations, and updates subsequent reasoning steps accordingly. ReAct's Act step enables real-time retrieval from external environments, directly mitigating hallucination.

Rondon et al. [51] demonstrated the viability of agent-based APR and the value of rich bug reports to produce fixes in industrial benchmarks by introducing Passerine, a dynamic ReAct agent, and GITS-Eval, a refined benchmark of 178 Google bugs. Using Gemini 1.5 Pro with only 5 simple commands and 20 independent trajectories per bug, Passerine fixed 73% of machine-reported, while the figure for human-reported bugs was only 25.6%.

Aligning with the ReAct framework, RepairAgent [4] deploys GPT-3.5 as an autonomous agent that iteratively plans and executes repairs. It constructs dynamically augmented prompts to facilitate a continuous cycle of reasoning about a bug and acting upon it using 14 available tools. The agent successfully fixed 164 Defects4J bugs, averaging 35 cycles per bug while keeping costs modest at $0.14 per fix.

These studies demonstrate that ReAct prompting can effectively resolve a significant portion of bugs in large-scale industrial environments without fine-tuning. However, this capability requires high computational costs due to frequent LLM invocations and repeated test executions.

Table C.9 in Appendix C details the individual approaches, including their retrieval strategies, prompt compositions, and reported metrics.

3.3. Quality Assurance and Reliability-based Techniques

This section explores strategies designed to ensure patch correctness, focusing primarily on post-generation validation, test execution, and iterative refinement loops.

3.3.1. Prompt Chaining

Prompt chaining decomposes complex, multi-step problems into a pipeline of smaller, interconnected sub-prompts, where the output of one step serves as the input for the next. Unlike monolithic prompting, this modular architecture facilitates the integration of external tools (e.g., compilers, fault localizers) and isolates independent logical flows.

AutoCodeRover [81] exemplifies this strategy by autonomously resolving GitHub issues through a two-stage decomposition. The first agent retrieves relevant context, while the second agent synthesizes the patch based on that retrieval. When evaluated using GPT-4, AutoCodeRover achieved a 19% pass@1 rate on the SWE-bench-lite (Python) benchmark. Notably, it averaged only 195 seconds and $0.43 per task, demonstrating that chaining can offer a superior cost-efficiency ratio compared to exhaustive baselines.

Robin [3] employs four specialized agents, Responder, Collaborative, Follow-up, and Context Retrieval, which alternate between automated reasoning and human guidance through an "investigate-and-respond" process. This architecture yields a 2.5× improvement in bug localization and 3.5× increase in bug resolution success, although its effectiveness remains contingent on the capabilities of the backbone model.

Similarly, PATCH [80] simulates a software development lifecycle by chaining multiple agents acting as testers, developers, and reviewers. The prompts include role-specific instructions enriched with code dependency analysis and programmer intent. Through this multi-stage feedback loop, PATCH achieved a Fix@1 score of 33.97% on the augmented BFP dataset, successfully repairing 283 unique bugs without fine-tuning.

Inspired by cognitive theories of debugging, UniDebugger [29] introduces an end-to-end, hierarchical multi-agent framework that structures test-driven debugging into seven specialized LLM agents organized across three workflow levels. The framework adopts a staged execution strategy: lightweight agents are applied first, and more expensive reasoning, tool use, and

external retrieval are invoked only when earlier stages fail. Evaluated on three established APR benchmarks, UniDebugger reports notable gains over prior approaches.

T^3 [35] extends this line of work by decomposing repair into four sequential dependent stages: BM25-based retrieval, cause analysis, repair planning, and patch generation, with a nested Forest of Thinking mechanism that integrates tree-based expansion and self-consistency voting within the intermediate stages. On the MODIT benchmark, T^3 improves repair rates over standard CoT by 11.2% and 9.9% on B2Fs and B2Fm respectively, while also outperforming Tree-of-Thought, Plan-and-Solve, and Analogical Reasoning. However, the framework exhibits lower self-consistency accuracy on the more challenging B2Fm subset, indicating a trade-off between exploratory reasoning breadth and convergence toward stable outputs.

Abdinabiev et al. [1] propose a multi-agent orchestration in which three agents exchange information through a shared, template-structured context pool, combining retrieval-augmented context with iterative, feedback-conditioned prompt refinement. Evaluations on Defects4J and SWE-Bench Lite demonstrate improved repair performance across both proprietary and open-source LLMs, particularly for complex multi-function bugs. However, the results also show that dynamic context contributes relatively little for stronger models, such as GPT-4o, as approximately 94% of correct repairs are produced in the first iteration using only the initial static context, with iterative context refinement primarily benefiting more challenging defects and weaker models.

These studies collectively demonstrate that modeling repair as a collaborative, multi-step chain significantly enhances LLM performance in complex SE tasks.

3.3.2. Self-Consistency Prompting

Self-Consistency [59] enhances the reliability of CoT reasoning by replacing greedy decoding with a stochastic sampling approach. The model generates multiple diverse reasoning paths for the same problem, and the final answer is selected via majority voting or aggregation.

Ahmed et al. [2] adapted Self-Consistency for program repair by utilizing developer commit messages as CoT "reasoning paths" within few-shot examples. They prompted the LLM to generate a high volume of patch candidates via high-temperature sampling and applied majority voting to determine the most frequent solution. This approach surpassed all prior methods on the MODIT dataset, achieving a 42.1% improvement in exact-match accuracy.

While self-consistency offers robustness, it introduces distinct trade-offs. The technique's redundancy filters out stochastic failures and identifies reliable solutions in ambiguous scenarios. However, this benefit comes at a linear increase in computational cost, as the approach requires multiple forward passes through the model for every single bug. Furthermore, the majority voting mechanism risks amplifying common misconceptions if the model is systematically biased toward a specific incorrect pattern.

3.3.3. Prompt Tuning

Distinct from the discrete (text-based) prompting strategies discussed thus far, Prompt Tuning [30] sits at the intersection of prompting and Parameter-Efficient Fine-Tuning (PEFT). In this paradigm, the LLM remains frozen, and optimization is performed on a small sequence of continuous vector embeddings (soft prompts) prepended to the input. These trainable vectors condition the model for specific tasks without the massive resource overhead of full parameter updates.

Cai and Jiang [6] conducted the first evaluation of prompt tuning for APR, specifically targeting data-scarce scenarios. By augmenting soft prompts with domain-specific code knowledge, their approach outperformed traditional fine-tuning across six datasets and four programming languages, achieving an average improvement of 87.33%.

These results position prompt tuning as a flexible alternative to conventional fine-tuning, capable of injecting domain knowledge directly into the repair process. However, its classification as a "prompting" technique remains debated, as it requires a training phase and hyperparameter optimization. Furthermore, its efficacy relies heavily on the quality of the injected knowledge, where incomplete or noisy data can destabilize the soft prompts. Consequently, broader adoption hinges on developing more robust parameterization methods.

Table D.10 (in Appendix D) details all studies in this category, summarizing their prompt chaining mechanisms, LLM backbones, and empirical results.

4. Findings

4.1. Synthesis Methodology

To clarify how our "Insights" and "Key Trends" were derived, we outline our synthesis methodology. Due to high heterogeneity in benchmarks, metrics, and experimental settings across the 53 primary studies, a quantitative meta-analysis was not methodologically defensible. We therefore adopted a structured narrative synthesis combined with vote counting—a standard practice for heterogeneous SLRs [27]. Our process comprised three steps:

1. **Extraction:** We iteratively open-coded the five dimensions from Section 2.4 into Tables B.8, C.9, and D.10.

2. **Cross-Tabulation:** We tabulated prompting-strategy families against outcome types (e.g., correct/plausible patches, pass@k), target benchmarks/bug types, and evaluation assumptions (e.g., perfect fault localization). All quantitative statements reflect frequency counts across these tabulated dimensions.

3. **Synthesis:** We derived qualitative insights by comparing studies sharing common benchmarks and outcome metrics, prioritizing primary within-study ablations for causal claims and explicitly flagging authorial interpretations.

All coding was conducted independently by two authors and reconciled through discussion with a third.

4.2. Temporal Evolution

As Figure 3 illustrates, the field has grown unevenly, driven by a post-2024 surge in "Cognitive and Reasoning Enhancement" literature. Notably, 2025 already records the highest publication volume, indicating accelerating research momentum. Rather than representing isolated annual counts, the distribution should be interpreted as a temporal trajectory that reveals three distinct phases, each associated with major advances in model capabilities and shifts in evaluation infrastructure.

The first phase (2022–2023) represents a baseline-validation period. All 2022 studies (two studies) belong to the Structure and Context-Based family, while 2023 remains dominated by this category (five of nine studies), with zero- and few-shot prompting prevailing. Research at this stage primarily assessed whether general-purpose LLMs could perform code repair under minimal prompting, relying on direct queries and limited prompt engineering. Reasoning-oriented and reliability-oriented strategies were largely absent or only emerging.

The second phase (2024–2025) marks a transition from feasibility assessment to structured control of the repair process. Cognitive and Reasoning Enhancement becomes the dominant family, increasing from two studies in 2023 to nine in 2024 and ten in 2025. This growth reflects both the emergence of stronger instruction-following models supporting multi-step reasoning and the rise of repository-level and multi-file benchmarks that exposed the limitations of single-pass prompting. CoT (seven studies) and retrieval-augmented prompting (nine studies) drive this trend by improving reasoning guidance and grounding generated patches in relevant context. In parallel, Quality Assurance and Reliability-Based strategies expand (three studies in 2024, seven in 2025), with prompt chaining (ten studies overall) enabling complex repairs through sequential and verifiable interactions. Together, these trends show a shift toward combining reasoning enhancement with reliability mechanisms.

The third phase (2026) represents an early stage of consolidation. Cognitive and Reasoning Enhancement remains the largest category by count even at this early stage of the year, consistent with its dominance in 2025. Given the small sample size and the partial-year nature of the 2026 count, this distribution should be read cautiously rather than as confirmation of a settled trend; it is equally consistent with a publication lag in pipeline-oriented work as with an early sign that reasoning-enhancement techniques continue to dominate research attention while reliability-focused contributions take longer to mature into publishable results.

Across the three phases, Structure and Context-Based prompting has become a validated baseline, Cognitive and Reasoning Enhancement remains under active refinement, and reliability and combination approaches represent the emerging frontier, with ongoing efforts to determine when their additional complexity is beneficial.

Overall, the literature indicates a gradual move from isolated prompting strategies toward integrated multi-prompting workflows. The combination of reasoning, contextual retrieval, and iterative feedback reflects an emerging consensus that no single prompting strategy is sufficient for complex repair tasks. Future advances are therefore likely to rely on adaptive prompt orchestration rather than fixed prompting patterns.

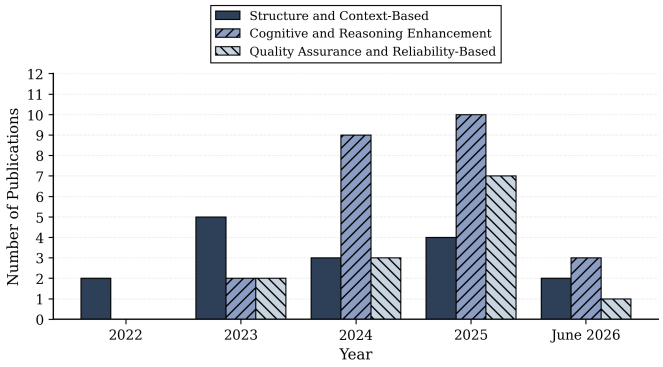


Figure 3: Evolution of prompting strategy families over time (January 2022-June 2026).

Besides, the distribution of studies across publication venues (Figure 4) shows that LLM-based software engineering research is primarily concentrated in leading software engineering outlets. ICSE, ISSTA, PACMSE, and ASE remain the main publication venues, with ICSE reaching peak activity in 2025, and ISSTA in 2024. PACMSE first appears in 2025 and remains active in the half-year 2026 dataset, contributing two studies in each year. The ASE journal, distinct from the ASE conference, emerges only in 2025 and accounts for two studies. Overall, conferences remain the dominant dissemination channel, with 37 of the 53 studies published in conference venues compared with 16 in journals. The Others category maintains a relatively stable share across years rather than showing a clear upward trend. The 2026 count reflects half-year data and should therefore not be interpreted as a decline in publication activity.

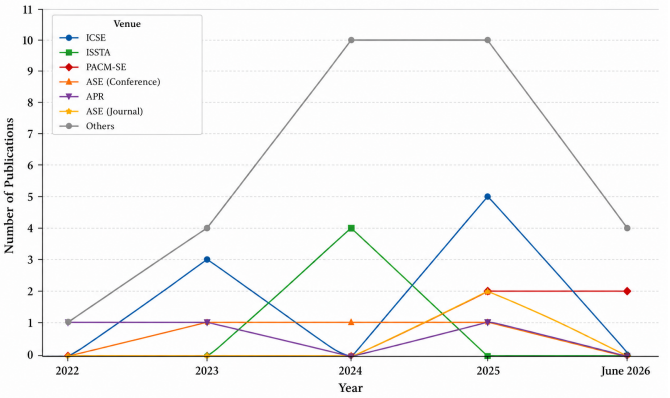


Figure 4: Distribution of selected studies across major publication venues from January 2022 to June 2026.

4.3. Impact of Prompting Strategies on Repair Performance

The relationship between prompting strategies and repair performance across bug types reveals both systematic patterns and context-sensitive variations. While structured prompting generally improves repair outcomes for decoder-only models

generating entire functions, this benefit does not extend to encoder-decoder or infilling models, where structured output is already the default training objective. Furthermore, structure alone is not the determining factor; the interaction between prompt design, retrieval mechanisms, and model capacity is critical. The reported studies provide enough comparative data to estimate the approximate magnitude of these effects, even though they were not designed as a unified benchmark.

Zero-shot prompting serves primarily as a baseline across studies rather than a competitive strategy. Base LLM performance without structured guidance is consistently low, with reported correct fix rates ranging from 15% to 67.8%. The evidence from the literature consistently demonstrates that zero-shot prompting, while computationally inexpensive, underperforms more structured approaches on both function-level repair tasks and coding challenge benchmarks, but the magnitude of the gap varies substantially by task and model. A model may generate plausible-sounding but incorrect explanations that lead to faulty repair decisions. These findings suggest that zero-shot performance depends on the clarity of the code context and degrades substantially when the required repair context extends beyond a single function.

Reasoning-based techniques such as CoT are particularly effective at resolving semantic and logical errors, and the size of this effect can be approximated from studies that report paired comparisons. CoT tends to improve APR performance relative to unstructured zero-shot baselines for most models, with reported gains ranging from +6% to +377% depending on the model, benchmark, and fact-set composition. However, this benefit is not guaranteed: it is model-dependent (e.g., CoT degrades performance for reasoning-optimised models such as o1-mini) and can be offset by excessive or irrelevant context. The improvement is most consistent for open-weight models and general-purpose proprietary models on the Defects4J and QuixBugs benchmarks, but on simpler benchmarks (e.g., ARHE) the advantage may be negligible.

Similarly, ToT approach outperforms linear prompting on the same benchmark, a relative improvement of roughly 30% attributable to exploring and selecting among alternative reasoning paths rather than committing to a single reasoning chain. This gain is not unconditional, however: repository-level evaluation shows that Chain-of-Thought can reduce GPT-4's performance on interface inconsistency bugs, where flawed intermediate reasoning steps tend to propagate into the generated patch rather than being caught before code is produced.

Few-shot prompting with retrieved or generated demonstrations consistently outperforms zero-shot baselines across the reviewed studies, though its effectiveness depends heavily on the relevance of the selected examples. Across these studies, demonstration quality and query relevance matter more than sheer quantity, making the retrieval or generation of task-relevant examples an effective strategy. Notably, few-shot examples primarily improve efficiency, reducing the number of repair attempts needed, rather than effectiveness, as they do little to expand the overall set of fixable bugs.

RAG demonstrates promising yet context-dependent performance gains. Empirical evidence suggests that its relative benefit is inversely associated with model size and parametric knowledge. Smaller models benefit more substantially from RAG because they possess less parametric knowledge about relevant coding problems and can leverage non-parametric knowledge to provide accurate fixes. For larger models, the improvement is smaller but still notable. This suggests a diminishing returns relationship: as models become more capable, the marginal benefit of external retrieval decreases, though it remains non-zero even for large models. However, retrieval quality is a more significant bottleneck than prompt sophistication. Failures in retrieval, whether due to unimplemented search APIs, low similarity matches, or misleading issue descriptions substantially degrade repair outcomes, even when the prompting strategy itself is well-designed. This suggests that improving code search and context retrieval may yield larger gains than refining prompts for the same context, and that example selection is bounded by the quality of the underlying diagnostic information used for retrieval.

Empirical evidence consistently indicates that the first iteration of conversational repair yields the largest performance gain, with subsequent turns producing diminishing returns and, in some cases, introducing refusal behavior, repetitive outputs, or outright degradation. This finding challenges the intuitive assumption that "more conversation leads to better outcomes," suggesting instead that prompting strategies should be designed to maximize information gain in early turns rather than relying on iterative refinement to converge toward a correct solution. Moreover, the effectiveness of conversational prompting depends critically on the quality and structure of the feedback provided, rather than merely its volume. Adding more information does not linearly improve repair outcomes; the form of the information matters more than its quantity. For instance, contrastive test pairs consistently outperform simple test-failure information, despite both being categorized as "conversational" feedback. Similarly, not all forms of feedback are equally beneficial: the specificity and nature of the feedback (e.g., an error message versus a location hint) moderates the magnitude of improvement observed.

Bug type sensitivity patterns emerge across multiple dimensions, with multi-line and multi-function bugs exhibiting divergent responses to prompting sophistication. To systematically capture these cross-tabulation dynamics at the level of individual bug categories, Table 3 synthesizes empirical findings across fifteen primary bug categories, mapping prompting strategies and model architectural paradigms to category-specific repair outcomes.

For logical and semantic defects, zero-shot frontier models frequently fail on misaligned algorithms and multi-location errors due to patch search space limitations. CoT prompting mitigates these bottlenecks for standard proprietary and open-weights models by structuring step-by-step reasoning, although missing function calls remain persistent failure modes. Notably, explicit CoT instructions produce counterproductive effects on native reasoning architectures, where external step-by-step prompts interfere with intrinsic reasoning mechanisms. Conversely, boundary and corner-case defects respond reliably to both CoT and prompt chaining because localized off-by-one

logic aligns directly with targeted reasoning steps.

System-dependent defects, including network, GUI, and performance issues, prove sensitive to context depth rather than strategy elaboration alone. Incorporating repository- and project-level context into CoT prompts yields substantial gains across open-weights and proprietary models by exposing structural dependencies beyond local function scope. Among interface inconsistencies, parameter order and return context mismatches remain significantly harder to fix than simple count mismatches. For these structural interface alignments, concise one-shot prompting outperforms both zero-shot and CoT approaches, as a single representative example provides immediate structural templates while avoiding the extraneous reasoning overhead that degrades CoT performance.

Defects identifiable through static analysis, such as runtime exceptions, concurrency violations, resource leaks, and type errors, achieve peak resolution through retrieval-augmented prompting and domain-specific CoT prompting. Combining static annotations with retrieved fix patterns or internal donor code guides models toward defensive checks and synchronization controls, although dynamic execution feedback remains necessary to prevent secondary assertion failures. Finally, while RAG effectively resolves compilation errors across multiple languages, syntax defects remain underrepresented in standard APR benchmarks despite their frequency in general software development.

Overall, prompting effectiveness depends on the alignment between strategy, model characteristics, and bug type. While added structure (e.g., reasoning, retrieval, context) can improve repair, its benefits are conditional; for instance, CoT helps general-purpose and open-weight models but may hinder reasoning-optimized models and interface bugs.

4.4. Impact of Model Characteristics on Prompt-based Program Repair

GPT family dominance: OpenAI's GPT models (GPT-3.5 and GPT-4 variants) are employed in 35 studies (66% of the corpus), frequently as the sole evaluated model or as a reference baseline for comparison. This makes the GPT family the most prevalent choice among the analyzed studies for code intelligence tasks. This prevalence may be associated with several factors, including their competitive benchmark performance, strong instruction-following capabilities, and broad accessibility through commercial APIs. However, this reliance on proprietary services may introduce reproducibility challenges due to limited access to model implementations and training details.

Among these 35 papers, only five report a direct comparison between GPT-3.5 and GPT-4 within the same experimental setup. A synthesis of the five studies reveals that GPT-4 consistently outperforms GPT-3.5 across most benchmarks and prompting strategies, with the performance gap attributed to enhanced reasoning, context integration, and intermediate task accuracy. However, the magnitude of this advantage is contingent on benchmark complexity and prompt design; simpler tasks like QuixBugs narrow the gap and a direct reversal was observed on HumanEval-Java with trace-based prompts, where GPT-3.5 achieved higher Correct Program Accuracy. The divergence stems from the interplay between model capability and the nature of the task: GPT-4's larger context and reasoning prowess are more beneficial for repository-level bugs and complex traces, while its advantage diminishes on algorithmic or synthetic defects where additional information may distract rather than aid. These findings suggest that GPT-4 is not merely better on average, but more robust when APR prompts require it to filter repository or repair context, reason through harder bug structures, or make use of iterative or dynamic feedback, whereas GPT-3.5 benefits most in settings where the prompt scaffold simplifies the repair search rather than enriching it with difficult evidence.

Specialized Code Models: Code-specialized models including Codex, CodeLlama, CodeT5, StarCoder, and DeepSeek-Coder collectively appear in 32 studies. This confirms that while GPT-family models dominate overall, explicitly code-specialized LLMs still constitute a substantial portion of APR research, accounting for roughly half of the frequency of GPT-3.5 or GPT-4 families individually. This reflects a strong interest in models pre-trained specifically on source code rather than general-purpose language models. This trend suggests recognition that domain-specific pre-training is more predictive of repair performance, as domain-specific coding models outperform general-purpose models of similar size.

Model diversity: We identified 20 distinct model families employed across the reviewed papers, demonstrating active exploration beyond the most popular choices. This diversity indicates a healthy research ecosystem that is not overly dependent on a single model provider or architecture. Table 4 categorizes models by family and reports their frequency across studies.

Open-source versus proprietary models: Proprietary API-based models (e.g., Claude, GPT-4, Codex) account for the majority of model usage in the analyzed corpus (63.6%). This observation should not be interpreted as a consequence of access modality alone, but rather as the result of multiple interacting factors. API availability reduces deployment complexity by eliminating the need for local GPU infrastructure and allows researchers to focus on prompt design and repair strategies. In addition, proprietary models have benefited from reported capabilities in instruction following, competitive performance on code-related benchmarks, and widespread adoption within the research community. Conversely, open-source models (e.g., CodeT5, CodeLlama) represent 36.4% of the analyzed usage and are increasingly explored due to their transparency, reproducibility, and potential for customization. Since most primary studies do not explicitly justify their model selection criteria, the relative influence of these factors cannot be conclusively determined from the reviewed corpus.

Taken together, these findings suggest a methodological tension in the field. The dominance of proprietary models in terms of usage frequency raises reproducibility concerns, since these models are subject to silent updates, deprecation, and usage costs that can undermine the long-term replicability of reported results. The growing, though still secondary, adoption of open-weight alternatives points to an emerging counter-trend toward reproducible, cost-free, and locally verifiable experimentation. We argue that this shift is likely to accelerate as open-weight models continue to close the performance gap with proprietary

Table 3: Bug-category-level repair performance by prompt strategy and model architecture

Bug Category	Prompt Strategy	Model Architecture	Metric	Reported Value	Primary Studies
Logical / Semantic	Zero-Shot	[PF] (Codex)	# Correct Patches	16	Fan et al. (2023) [14]
	CoT	[OW] (DeepSeek-V3)	# Plausible Patches	30/33	Darwiyanto et al. (2025) [10]
	CoT	[PF] (GPT-4o)	# Plausible Patches	29/33	Darwiyanto et al. (2025) [10]
	CoT	[PF] (GPT-4-turbo)	—	N/A	TypeRepair (2026) [79]
	Prompt chaining	[PF] (GPT-4)	Success Rate (%)	100.0 (8/8)	ROBIN (2024) [3]
Syntactic / Compilation	Zero-Shot	[PF] (Codex)	# Correct Patches	0/37	Fan et al. (2023) [14]
	RAG	[PF] (Codex)	Pass@1 (%)	Python 94.0; C 63.0	Ring (2023) [25]
Boundary / Corner Cases	CoT	[OW] (DeepSeek-V3)	# Plausible Patches	7/7	Darwiyanto et al. (2025) [10]
	CoT	[PF] (GPT-4o)	# Plausible Patches	7/7	Darwiyanto et al. (2025) [10]
	Prompt chaining	[PF] (GPT-4)	Success Rate (%)	75.0 (6/8)	ROBIN (2024) [3]
Network	CoT	[OW] (Llama-3.3)	Repair rate (%)	78.72	Ehsani et al. (2025) [13]
	CoT	[PF] (GPT-4o-mini)	Repair rate (%)	80.85	Ehsani et al. (2025) [13]
Runtime Error	CoT	[PF] (GPT-4-turbo)	—	N/A	TypeRepair [79]
	RAG	[PF] (GPT-3.5-turbo)	—	N/A	DSRepair (2025) [46]
GUI-Related	CoT	[OW] (Llama-3.3)	Repair rate (%)	82.14	Ehsani et al. (2025) [13]
	CoT	[PF] (GPT-4o-mini)	Repair rate (%)	75.00	Ehsani et al. (2025) [13]
Concurrency	CoT	[PF] (GPT-4-turbo)	—	N/A	TypeRepair (2026) [79]
	RAG	[PF] (Codex)	Top-1 EM accuracy (%)	Java 77.4; C# 82.5	InferFix (2023) [24]
Performance	CoT	[OW] (Llama-3.3)	Repair rate (%)	87.0	Ehsani et al. (2025) [13]
	CoT	[PF] (GPT-4o-mini)	Repair rate (%)	81.0	Ehsani et al. (2025)
API Misuse	CoT	[PF] (GPT-4-turbo)	—	N/A	TypeRepair (2026) [79]
Permission / Deprecation	CoT	[OW] (Llama-3.3)	Repair rate (%)	50.0	Ehsani et al. (2025) [13]
	CoT	[PF] (GPT-4o-mini)	Repair rate (%)	50.0	Ehsani et al. (2025) [13]
Resource Leak	RAG	[PF] (Codex)	Top-1 Accuracy (%)	Java 71.2; C# 57.0	InferFix (2023) [24]
Null Pointer Dereference	RAG	[PF] (Codex)	Top-1 Accuracy (%)	Java 59.5; C# 66.7	InferFix (2023) [24]
Interface Inconsistency	Zero-Shot	[PF] (GPT-4)	Accuracy (%)	73.39	RLCE (2024) [9]
	One-Shot	[PF] (GPT-4)	Accuracy (%)	78.23	RLCE (2024) [9]
	CoT	[PF] (GPT-4)	Accuracy (%)	72.58	RLCE (2024) [9]
Assertion Error	RAG	[PF] (GPT-3.5-turbo)	—	N/A	DSRepair (2025) [46]
Type Error	RAG	[OW] (CodeT5)	# Correct Patches	62	RetypeR (2024) [16]

Note: [PF]: Proprietary Frontier; [OW]: Open-Weight; N/A: No data reported.

systems, and we encourage future program-repair studies to report results on at least one open-weight model to support independent verification and long-term comparability.

Architecture: The reviewed papers predominantly examine decoder-only models (e.g., GPT-4, GPT-3.5), alongside a smaller number of encoder-decoder (CodeT5) and encoder-only (CodeBERT) architectures. While architectural differences intersect with task formulation, they are rarely isolated as independent variables. For instance, Xia et al. [66] demonstrate that bidirectional infilling models outperform generative models on single-line repairs. However, Codex, a decoder only model with suffix infilling, achieved the highest overall performance. This suggests that scale and code centric pre-training may outweigh pure architectural class. Similarly, Xu et al. [70] argue that the infilling objective misaligns with decoder-only next-token prediction. Their D4C approach improves outcomes by reformulating repair as complete function generation rather than masked-span prediction, though it remains unclear whether these gains stem from task alignment, architectural suitability, or the GPT-4 as the backbone model.

Model size: Model size emerges as a consistently influential factor, with several studies reporting scaling effects, though the relationship is neither linear nor uniform. Xia et al. [66] demonstrate monotonic improvements within the GPT-Neo family (125M to 2.7B), GPT-J (6.7B), and GPT-NeoX (20B). Hu et al. [19] observe comparable trends across GPT-family models, with O1-preview and O1-mini repairing all 40 QuixBugs bugs, outperforming GPT-4o (38/40) and ChatGPT (31/40). However, Codex (12B) outperforms the larger GPT-NeoX (20B), a reversal [66] attribute to task-specific code generation fine-tuning—a finding echoed by [10], where DeepSeek-V3 (670B) achieves the highest plausible patch count (36.6 on QuixBugs) despite being evaluated alongside smaller models. Collectively, these results suggest raw parameter count interacts with training objectives and data in ways not yet well understood. Furthermore, inference costs are rarely weighed against marginal gains. For example, GPT-NeoX generates patches 71× slower than GPT-Neo 125M but yields a disproportionately small increase in fixed bugs. This scale efficiency tension remains acknowledged but poorly analyzed for practical deployment trade offs.

These model-level patterns point toward an interaction rather than a set of independent effects. The Codex-versus-GPT-NeoX reversal and DeepSeek-V3's strong plausible-patch count despite its scale suggest that training objective and code-specific finetuning shape repair performance at least as strongly as parameter count, while the GPT family's dominance in the corpus means that many of the prompting-strategy findings in Section 4.2 have been validated on a narrow slice of available models. Because model choice and prompting strategy are rarely varied independently within the same study, the corpus does not yet support a clean separation of how much of a given repair result comes from the model itself versus the prompting strategy applied to it, and we treat this as a limitation of the current evidence base rather than a settled finding.

Although proprietary general-purpose models dominate current APR research, raw model size does not dictate performance. Specialized fine-tuning and training objectives prove far more decisive, allowing smaller code-tuned models to surpass larger general ones. However, performance attribution remains confounded because studies rarely isolate model selection from prompting strategy.

4.5. Analysis of Evaluation Benchmarks

The reviewed studies rely on a highly concentrated set of evaluation benchmarks, with empirical validation focused on only a few datasets. In practice, QuixBugs and Defects4J dominate experimental evaluation, appearing in a substantial portion of the literature. Many studies pair these benchmarks within the same experimental setup to suggest broader applicability across bug collections, particularly in mixed Java–Python settings. Figure 5 illustrates the distribution of evaluation benchmarks used across the reviewed studies, along with the number of works adopting each benchmark.

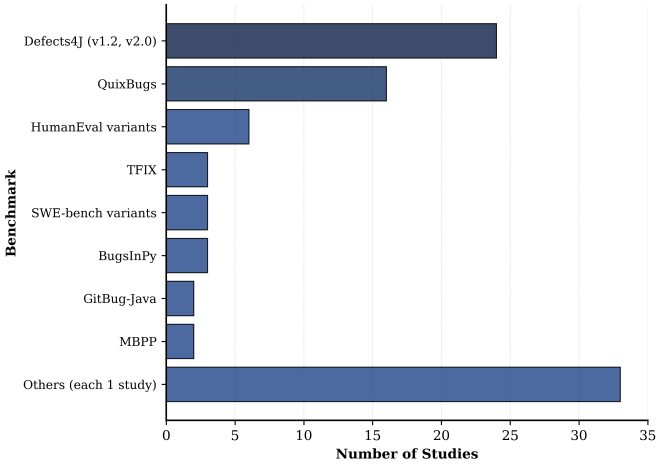


Figure 5: Distribution of the evaluation benchmarks.

Defects4J dominance: Defects4J dominates as the most frequently used benchmark for evaluating Java-based APR approaches, appearing in 29 studies (54.7%) across versions 1.2 and 2.0. Its dominance reflects its repository of real-world bugs, extensive test suites, and well-established evaluation protocols that collectively ensure replicability and comparability across studies.

QuixBugs prevalence: QuixBugs is utilized in 18 studies (34%), serving as a complementary benchmark to Defects4J. While Defects4J focuses on bugs from large-scale real-world systems, QuixBugs provides a collection of algorithmically challenging bugs that are relatively small in scope. Notably, QuixBugs supports cross-language evaluation by providing parallel implementations in both Java and Python, enabling researchers to assess the language-agnostic capabilities of repair techniques.

Code generation benchmarks: HumanEval and MBPP, originally designed for code generation tasks, collectively appear in eight studies, indicating that some researchers conceptualize program repair as a generation problem. However, this adaptation raises concerns about benchmark suitability, as code generation benchmarks may not adequately capture repair-specific

Table 4: LLM backbone usage frequency

Model Family	Specific Models	Study Count	Studies
OpenAI GPT-3.x	GPT-3, GPT-3.5, GPT-3.5-turbo	20	ChatRepair (2024) [68], ThinkRepair (2024) [76], RepairAgent (2025) [4], ContrastRepair (2025) [28], CREF (2024) [72], UniDebugger (2025) [29], Wuisang et al. (2023) [64], Omari et al. (2024) [44], Do et al. (2023) [12], LDB (2024) [83], Haque et al. (2025) [17], Cao et al. (2025) [7], Maniple (2025) [48], CodeCorrector (2025) [32], CodeFixer (2024) [33], RLCE (2024) [9], DSrepair (2025) [46], PATCH (2025) [80], TypeRepair (2026) [79], T^3 (2025) [35]
OpenAI GPT-4.x	GPT-4, GPT-4-turbo, GPT-4o, GPT-4o-mini	23	AutoCodeRover (2024) [81], CREF (2024) [72], UniDebugger (2025) [29], Hu et al. (2025) [19], D4C (2025) [70], Qu et al. (2024) [50], Haque et al. (2025) [17], CodeCorrector (2025) [32], Darwiyanto et al. (2025) [10], DRCodePilot (2024) [82], REx (2024) [57], RLCE (2024) [9], DSrepair (2025) [46], Robin (2024) [3], AdverIntentAgent (2025) [75], RGD (2024) [23], SpecRover (2025) [52], Ehsani et al. (2025) [13], TypeRepair (2026) [79], DeepK (2026) [31], T^3 (2025) [35], Agentless (2025) [65], Abdinabiev et al. (2026) [1]
Meta LLaMA	CodeLlama, LLaMA2, Llama3, Llama3.3	12	ThinkRepair (2024) [76], CREF (2024) [72], UniDebugger (2025) [29], LDB (2024) [83], Maniple (2025) [48], Darwiyanto et al. (2025) [10], RAGFix (2024) [39], ReAPR (2025) [36], Ehsani et al. (2025) [13], Liu et al. (2026) [34], HAFix (2026) [55], Abdinabiev et al. (2026) [1]
OpenAI Codex	Codex	6	Prenner et al. (2022) [49], Fan et al. (2023) [14], Xia et al. (2023) [66], Cedar (2023) [42], InferFix (2023) [24], RING (2023) [25]
DeepSeek	DeepSeek-Coder, DeepSeek-V3, DeepSeek-Coder-V2	9	ThinkRepair (2024) [76], UniDebugger (2025) [29], Darwiyanto et al. (2025) [10], DSrepair (2025) [46], ReAPR (2025) [36], Liu et al. (2026) [34], HAFix (2026) [55], DeepK (2026) [31], ExpeRepair (2026) [41]
Google	PaLM2, Gemini-1.5-Pro, Gemini-1.5-Flash, Gemini 1.5 Pro, Bard	5	CREF (2024) [72], UniDebugger (2025) [29], Darwiyanto et al. (2025) [10], Passerine (2025) [51], RLCE (2024) [9]
Anthropic Claude	Claude-instant, Claude 3.5 Sonnet	5	CREF (2024) [72], Unidebugger (2025) [29], SpecRover (2025) [52], Agentless (2025) [65], ExpeRepair (2026) [41]
EleutherAI	GPT-Neo, GPT-J, GPT-NeoX	3	Xia et al. (2023) [66], ReAPR (2025) [36], Cai et al. (2025) [6]
OpenAI ChatGPT	ChatGPT	3	AutoSD (2025) [26], Hu et al. (2025) [19], Sobania et al. (2023) [56]
Salesforce	CodeT5	3	Xia et al. (2023) [66], RetypeR (2024) [16], Cai et al. (2025) [6]
OpenAI o1	o1-preview, o1-mini	3	[19], Darwiyanto et al. (2025) [10], ExpeRepair (2026) [41]
StarCoder	StarCoder	2	ThinkRepair (2024) [76], LDB (2024) [83]
Qwen	QwenCoder	2	Liu et al. (2026) [34], Abdinabiev et al. (2026) [1]
Other	-	1 each	Varies

challenges such as comprehending existing code context, localizing faults, and minimizing modifications to preserve program structure and style.

Repository-level evaluation: SWE-bench appears in three studies and represents a notable shift toward evaluating repository-level repair capabilities. Unlike traditional benchmarks that focus on isolated bugs within single files, SWE-bench requires approaches to understand natural language issue descriptions, navigate complex codebases, and coordinate changes across multiple files. This evolution reflects the growing recognition that practical program repair must address real-world complexity beyond single-file bugs.

Benchmark fragmentation: The identification of 47 distinct benchmarks reveals substantial fragmentation within the evaluation landscape. While such diversity supports broader coverage of repair scenarios, it hinders direct cross-study comparison and raises concerns about benchmark overfitting and inconsistent methodological baselines.

Benchmark complexity effects: Performance varies considerably across benchmark types. Single-function, algorithmically-focused benchmarks (HumanEval, MBPP, QuixBugs) consistently achieve 64-100% success rates with advanced prompting strategies, while real-world, multi-hunk bugs (Defects4J, SWE-bench) demonstrate $15 - 40\%$ success rates even with sophisticated approaches. This 2-3$\times$ performance gap persists across all prompting paradigms, indicating fundamental differences in bug complexity rather than methodological limitations.

Test-Suite Quality: The relationship between reported repair effectiveness and the strength of the test oracle used for evaluation warrants explicit consideration. In the surveyed literature, high success rates on established benchmarks such as Defects4J are often presented as evidence of prompting strategy quality. However, this interpretation is doubly conditioned on test suite characteristics: dense test suites with rich execution logs simplify the repair task through feedback, whereas sparse test suites risk accepting semantically incorrect patches that merely overfit available assertions.

Despite a field-wide tendency to treat test suites as unexamined ground truth, our systematic analysis identified four studies that explicitly engage with overfitting as a core phenomenon rather than an unmeasured byproduct. From the perspective of agentic and adversarial filtering, Abdinabiev et al. [1] implement a dedicated Overfitting Detector agent to audit candidate patches, while AdverIntent [75] introduces Agenttest, an adversarial test-generation mechanism specifically designed to detect and discard overfitted patches during the repair loop. In terms of disaggregating patch quality, AutoCodeRover [81] explicitly reports the performance gap between plausible (test-passing) and correct patches, analyzing how omissions in issue descriptions drive models toward overfitting. Finally, through qualitative mechanism analysis, D4C [70] provides a detailed worked example of a plausible-but-incorrect patch, dissecting the exact mechanism by which the language model satisfied test assertions while introducing semantically invalid logic.

While these studies demonstrate emerging sophistication in handling patch quality, such practices remain the exception across the broader literature. That four unrelated designs, preventive, corrective, causal, and comparative, independently confirm the same failure suggests overfitting is a structural limitation of test-based validation rather than an artifact of any single method.

Ultimately, reliance on Defects4J and QuixBugs has narrowly defined benchmark success in APR. The sharp drop in performance—where single-function success rates double or triple those on repository-scale, multi-hunk bugs—reveals that reported gains track structural defect complexity far more than prompting sophistication.

4.6. Analysis of Programming Language Coverage

As shown in Table 5, the analyzed studies exhibit a strong concentration on Python and Java, appearing in 40 and 26 studies respectively across the 53 studies reviewed. This concentration reflects both industrial adoption patterns and the availability of established bug benchmarks.

Systems languages remain markedly underrepresented: C appears in only 5 studies and C++ in 3, despite their prevalence in embedded and performance-critical software. The likely cause is the inherent complexity of low-level defects—memory management, concurrency, and pointer operations—which compound the challenges LLM-based repair models already face.

Table 5: Programming Language Distribution across Reviewed Studies

Programming Language	Study Count	Percentage (%)
Python	40	75.47
Java	26	49.06
C	5	9.43
JavaScript	4	7.55
C++	3	5.66
C#	2	3.77

Note: A single study may include multiple programming languages.

JavaScript, despite its web development dominance, is evaluated in just 4 studies. Its dynamic typing, asynchronous execution, and DOM manipulation semantics distinguish it substantially from the predominantly studied languages, representing a notable gap given JavaScript's prevalence.

Notably absent from the surveyed literature are Go, Rust, TypeScript, Swift, Kotlin, and PHP from the surveyed literature. Given Rust's rising adoption in systems programming and TypeScript's in web development, this absence represents a research gap and a limitation for broader APR adoption across diverse software ecosystems.

Ultimately, performance on a specific programming language depends heavily on its training distribution rather than overall model capability. Where language representation is low, prompting strategies cannot substitute for missing parametric knowledge, highlighting the necessity of retrieval-augmented approaches.

The reviewed evidence suggests that performance depends not only on overall model capability but also on the representation of the target language in training data. This finding highlights the importance of language-specific exposure in multilingual model selection. For under-represented languages, prompt-

ing strategies alone may be insufficient, and retrieval-based augmentation can provide valuable support.

4.7. Prompt Design

Across multiple studies, we consistently observe that moving from generic, instruction-only prompts to structured, context-rich templates yields increases in repair rates of up to two- to threefold in some studies, while more advanced strategies such as conversational refinement, adversarial reasoning, and output format alignment deliver additional improvement. This evidence suggests that prompt engineering has a direct impact not only on patch correctness and credibility, but also sample efficiency, generalization to complex bugs, and developer confidence.

The most basic enhancement is to add the contextual information as explicit prompts. Providing bug-aware context—failing tests, error messages, and domain-specific details—is beneficial in almost every case by allowing the model to concentrate on the real issue. Further than static context, iterative and conversational prompting introduces a second level of sophistication. The conversational aspect of ChatRepair [68], including compilation errors and test feedback from previous repair attempts, contributes 34 correct fixes beyond non-iterative use of GPT and verifies that learning from failed patches prevents redundant exploration. AdverIntent-Agent [75] encapsulates this intuition with an adversarial reasoning that encourages the model to produce a set of program intents and patches, achieving 19.4% improvement in patch generation by taking advantage of explicit exploration of diversified hypotheses. A third dimension of prompt effectiveness is to match the prompt structure with that in LLM's pre-training objective. Such effect of alignment is even more significant for decoder-only models, which tend to generate entire sequences of code rather than discrete fragments due to their causal language modeling objective.

Taken together, these findings show that designing effective prompts for APR is more than simply clever formatting. It requires aligning task framing with model architecture, managing context precision rather than maximizing context length, knowing when to ask for iterative feedback and turning predictability into explainability. The practice in use today is still mostly manual and model-specific with little theoretical understanding of why some prompt structures work better than others.

5. Challenges and Future Directions

Prompting-based APR has demonstrated measurable effectiveness, yet persistent limitations in reliability, scalability, and generalizability across programming contexts constrain broader adoption. The challenges and opportunities synthesized here outline a roadmap toward bridging the gap between experimental success and production-ready automated debugging systems.

5.1. Dependency on Perfect Fault Localization

A critical limitation in current LLM-based APR evaluation is the frequent assumption of perfect fault localization. Our analysis shows that 18 studies (34%) provide models with the exact buggy location(s), thereby removing localization uncertainty from the repair process. Since these studies eliminate a fundamental challenge of real-world debugging pipelines, their reported success rates should be interpreted as an idealized upper bound on repair capability rather than directly comparable estimates of end-to-end repair performance. Throughout this survey, we therefore distinguish results obtained with perfect fault localization from those relying on tool-based fault localization and avoid aggregating them into a single performance measure. Such aggregation would obscure the additional difficulty introduced by inaccurate localization, including false positives from real fault localization techniques and the increased risk of generating overfitting patches or introducing new defects.

To address this limitation, future work should evaluate prompting strategies in conjunction with top-k outputs from established FL tools, including spectrum-based approaches (e.g., GZoltar, Ochiai) and learning-based methods. We recommend reporting repair success rates at multiple top-k thresholds (k=1, 3, 5, 10) to better capture robustness to variations in fault localization quality. This stratified evaluation protocol provides a realistic view of LLM-based APR in practical debugging pipelines while establishing a consistent empirical basis for future surveys without relying on post-hoc normalization.

5.2. Multiple Function Constraint

The multi-function constraint captures a persistent limitation identified across the surveyed studies: the inability of many prompting strategies to effectively handle bugs whose repair spans multiple functions, files, or modules. While a substantial portion of the APR literature targets bugs localized to a single function, the approaches that succeed in these constrained settings frequently fail to maintain performance when the repair scope expands.

Our analysis reveals that 41.5% of the evaluated approaches restrict their validation exclusively to single-function bugs. This focus on localized repairs creates a disconnect between the evaluation conditions and the multi-file, cross-module defects that dominate industrial software maintenance. More critically, eight studies (15.1% of the corpus) explicitly documented performance degradation or outright failure when their prompting strategies were applied to multi-file bugs. These eight studies provide direct empirical evidence that the Multiple Functional Constraint is not merely a gap in evaluation coverage, but a substantive limitation of certain prompting approaches.

The failure patterns observed across these eight studies fall into two primary categories. The first category involves scope misinterpretation, where the model generates a repair that correctly addresses the local symptom within a single function but fails to propagate necessary changes to dependent functions or calling contexts. The second category involves context dilution, where the prompting strategy attempts to include multiple file contexts within the prompt window, but the model loses fidelity on the critical dependencies due to the increased token load or attenuated attention across distant code segments.

Future studies should gradually extend evaluation settings beyond single-function bugs, first to multi-function defects within a single file, and eventually to multi-file and cross-module cases, in order to better approximate the contextual and structural complexity of real-world software repair scenarios. We would also encourage future empirical work, and future syntheses of that work, to explicitly report scope-transfer results wherever they exist, since this is presently one of the thinnest evidentiary areas in the field despite its direct relevance to industrial applicability.

5.3. Benchmark Diversity, Generalizability, and Oracle Strength

The prevailing reliance on a small set of benchmarks (primarily Defects4J (54.7%) and QuixBugs (34%) poses validity and generalizability threats. Repeated evaluation risks benchmark overfitting, where prompting strategies and repair heuristics become implicitly tuned to specific bug patterns, undermining effectiveness on novel defects. Bug diversity is also inadequate: despite their well-curated design, both benchmarks fail to represent concurrency, performance, configuration, and security-related bugs. Furthermore, their strong test suites may artificially inflate repair success rates in ways that do not transfer to projects with weaker or incomplete test coverage. The community urgently needs to develop more diverse benchmarks that systematically cover underrepresented bug categories, including concurrency defects, performance regressions, and configuration errors.

Additionally, test-suite quality remains a persistent confounding variable across the surveyed literature. While recent agentic frameworks (e.g., Abdinabiev et al. (2026) [1], AdverIntent [75]) and analytical studies (e.g., AutoCodeRover [81], D4C [70]) actively attempt to detect or dissect overfitting, none of the surveyed papers systematically report formal coverage metrics (e.g., branch or path coverage) or mutation scores for the benchmark test suites. The adoption of curated benchmarks such as SWE-bench Lite-S and SWE-bench Verified represents meaningful progress toward reducing weak-oracle noise. Future studies should report coverage or mutation-score alongside success rates as a matter of course, and the field would benefit from a common minimal protocol, such as adversarial or held-out test validation, so that reported gains in prompting effectiveness can be disentangled from the strength of the test suite they were measured against.

5.4. Evaluation Metric Issues

Existing measurement practices suffer from inconsistencies that complicate fair comparison across APR studies. A key issue is the distinction between plausible and correct patches: plausible patches pass the available test suite but may still introduce regressions or fail on untested behaviors. Therefore, combining plausible and verified-correct patches can lead to misleading comparisons, as reported success rates may not represent the same notion of repair effectiveness. Our analysis shows that existing studies lack a shared protocol for separating these outcomes. Future evaluation frameworks should establish standardized reporting criteria that distinguish plausible from correct repairs before cross-study aggregation.

At the other extreme, exact match criteria are overly conservative, penalizing patches that are semantically equivalent to a reference implementation but differ syntactically. Across our synthesis, summarized in Appendix Tables 1 through 3, none of the reviewed studies report automated semantic-equivalence verification as part of their evaluation protocols. Instead, the reviewed corpus relies exclusively on exact match, plausible-patch counts based on test-suite passage, pass@k measures, or manual inspection by study authors. We identify this absence as an empirical finding of our survey, as it indicates that the field currently lacks a standardized approach for distinguishing genuine repair failures from patches that are functionally correct but syntactically different from the reference solution.

Beyond correctness, the field also lacks comprehensive evaluation of dimensions that matter for real-world deployment, including code maintainability, structural complexity, patch conciseness, and time efficiency. Across the surveyed studies, prompting strategies are evaluated almost exclusively through correctness or plausibility; where human-centric assessment does appear, typically through manual inspection or lightweight static analysis, patches are generally reported as syntactically consistent with surrounding code but occasionally non-idiomatic or unnecessarily complex. This asymmetry motivates a standardized evaluation framework combining (i) correctness validated against augmented test suites, (ii) plausibility assessment with explicit filtering criteria, (iii) quantitative measures of patch size and complexity, (iv) generation latency, and (v) human-centric qualities such as readability and maintainability, ideally assessed through studies involving human developers.

Finally, developing automated semantic-equivalence verification mechanisms, which our synthesis finds absent from the current corpus, represents an important future direction. Such mechanisms would enable more nuanced evaluation by recognizing functionally equivalent repairs despite surface-level syntactic differences and would provide a more consistent basis for interpreting exact-match failures in future APR studies.

5.5. Reproducibility and Determinism

The non-deterministic behavior of LLMs poses a fundamental reproducibility challenge for APR research. Identical prompts and input bugs can yield different patches across runs, undermining scientific validation. This variability complicates peer review and independent replication, as researchers may obtain inconsistent outcomes when reproducing published experiments, while also introducing substantial obstacles for benchmarking and comparative evaluation. Addressing non-determinism requires adopting several methodological practices. One direction involves configuring models with zero temperature and employing deterministic sampling settings, which can eliminate randomness during inference, ensuring identical outputs for repeated inputs. Alternatively, when leveraging the diversity of stochastic generation is desirable, generating multiple candidate patches followed by ensemble selection through voting mechanisms or consensus-based filtering can provide more stable and reliable repair outcomes while maintaining the benefits of exploring the solution space.

5.6. Cost Transparency and Efficiency

Computational cost remains largely underestimated in LLM-based APR, with only 15 of 53 studies (28.3%) providing explicit cost metrics, a gap that limits reproducibility and practical deployment. Multi-stage strategies such as prompt chaining, iterative self-refinement, conversational debugging, and ReAct-based reasoning require multiple LLM invocations per defect, and the trade-off between the resulting repair quality and the accumulated inference cost is not yet well understood at a field level.

Among the 15 studies reporting cost figures, the relationship between call volume and expense proves less straightforward than commonly assumed. Several high-invocation strategies still report modest per-bug costs: ReAct-style repair agents built on iterative tool-use loops average only cents to tens of cents per bug despite dozens of invocations per defect, while prompt-chaining pipelines generating roughly eighty candidate artifacts per bug report comparable costs alongside some of the higher resolution rates surveyed. This indicates that per-call pricing, backbone model choice, and prompt/context length can influence total repair cost as much as raw invocation count. The relationship between additional calls and repair quality is also non-linear. One retrieval-augmented pipeline that roughly tripled per-bug cost via additional candidate generations saw a comparatively small resolution-rate gain, suggesting diminishing returns beyond a certain iteration budget. Conversational and iterative self-refinement approaches span nearly an order of magnitude in per-bug cost, depending on conversation length, refinement-round limits, and underlying model family.

These findings offer preliminary evidence that multi-call strategies are not inherently impractical, but their economic viability is highly configuration-dependent and not predictable from strategy category alone. The lack of standardized cost reporting across the remaining 71.7% of studies, compounded by inconsistent definitions of a repair attempt, call, or iteration, still prevents rigorous cross-study comparison and complicates technique selection.

Taken together, these figures offer preliminary evidence that multi-call strategies are not inherently impractical, but their economic viability is highly configuration-dependent and cannot be inferred from the strategy category alone. The absence of standardized cost reporting across the remaining 71.7% of studies, together with inconsistencies in how the reporting studies define a repair attempt, a call, or an iteration, still precludes a more rigorous cross-study comparison and complicates technique selection at present.

To promote economic and computational transparency, we advocate for standardized reporting of: (i) complete token consumption statistics, covering both input and output tokens, per repair attempt, (ii) end-to-end latency measurements encompassing fault localization, prompt construction, patch synthesis, and validation phases, and (iii) explicit cost-performance trade-off analyses comparing resource-efficient approaches against their higher-cost counterparts. Such transparency would allow the preliminary patterns observed here to be tested more rigorously and would guide research toward automated repair solutions that are economically sustainable at the scale required for industrial adoption.

5.7. Context-window Limitation

Context window constraints remain the most persistent limitation in LLM-based APR, where fixed token budgets impose boundaries that undermine effectiveness in production-scale scenarios. Evidence from the RLCE [9] study indicates a measurable decline in repair accuracy for GPT-3.5 and GPT-4 as prompt length increases—a counterintuitive result explained by attention dilution, where transformer self-attention struggles to maintain focus on relevant elements as prompts lengthen, compounded by the recency bias of causal language models. Contemporary token limits—typically $4,096$ for GPT-3.5 Turbo and $8,000$ for GPT-4, frequently prove inadequate for repository-level repair, which demands reasoning over inter-file dependencies, cross-module API contracts, and global program state. The long-range dependency problem compounds this: even when context fits within the window, models struggle to propagate information between distant code regions. While emerging techniques, including intelligent slicing, syntax-hierarchy selection via `eWASH`, and incremental multi-turn context introduction, mitigate these pressures, they do not eliminate the underlying bottleneck.

Overcoming context constraints in LLM-based APR demands architectural innovations beyond simply extending window sizes, including better context compression, improved conversational summarization, and hybrid paradigms that integrate LLMs with traditional program analysis to supply structured semantic understanding beyond token-level context. Until these limitations are addressed, LLM-based repair systems are likely to remain constrained to single-component local bugs, continuing to struggle with multi-component, system-level defects, a class of faults that is generally more costly to resolve in real-world software engineering.

5.8. Prompt Security

As autonomous repair pipelines increasingly adopt multi-step reasoning frameworks (e.g., ReAct agents), prompt security emerges as a critical operational challenge. Because these agents interact directly with uncurated repositories—ingesting issue descriptions, code snippets, and commit histories—they are vulnerable to indirect prompt injection attacks. Adversarial text embedded within input artifacts can hijack an agent's control flow, forcing models to alter patch logic, bypass test verification, or execute unauthorized commands via tool interfaces. Although our primary focus remains the functional effectiveness of prompting techniques, practical deployment demands robust defense mechanisms. Future research must bridge the gap between prompt effectiveness and security by integrating input sanitization, least-privilege tool execution, and defensive context isolation into automated repair architectures.

6. Discussion

Our analysis reveals a clear hierarchical progression in prompting strategies for APR. While zero-shot and few-shot config-

urations remain foundational baselines, the landscape is currently dominated by Retrieval-Augmented Prompting for environmental grounding and Prompt-Chaining: the former grounds repairs in relevant code and documentation, while the latter decomposes complex fixes into guided reasoning steps. CoT and contextual prompting remain common for eliciting stepwise reasoning and preserving program context. Furthermore, we observe a trend toward agentic and iterative workflows, specifically through ReAct, conversational prompting, and self-refinement loops, which transition APR from static code generation to dynamic, tool-aware debugging. These advanced patterns not only enhance repair correctness but also improve traceability and facilitate developer-centric validation (**Answer to RQ1**).

Moreover, our analysis indicates that the studies collectively show that the effectiveness of prompting strategies is neither uniform nor bug-agnostic. While logical and semantic errors necessitate reasoning-oriented prompts to achieve correctness, simpler syntactic or structural issues are successfully resolved through low-overhead templates. Furthermore, we identify a 'scale-dependent threshold': while larger models (e.g., GPT-4) show significant performance gains from complex reasoning prompts, these same strategies often cause performance degradation or offer diminishing returns for models under 2.7B parameters, likely due to limited reasoning capacity (**Answer to RQ2**).

Furthermore, our analysis shows that although general-purpose GPT models dominate APR, effective repairs mainly depend on how well the model's architecture aligns with code-specific objectives. For instance, Codex-based architectures frequently outperform significantly larger general models due to their exposure to structured code semantics. Additionally, proprietary models are prevalent in research due to their instruction-following abilities, which make them more effective at handling complex, multi-step prompting strategies than open-source models of similar size (**Answer to RQ3**).

Taken together, RQ1–RQ3 reveal that prompting-based APR effectiveness arises from a three-way interaction among prompting strategy, model class, and bug category, rather than from any single factor in isolation (Table 3).

For logical and semantic defects, CoT benefits conventional models by explicitly structuring intermediate reasoning, but may hinder reasoning-native architectures (e.g., o1-mini) whose internal mechanisms already perform similar operations, making external scaffolding redundant or conflicting. For system-dependent defects (e.g., network, GUI, and performance issues), the primary bottleneck is not reasoning capability but contextual coverage; the observed benefits of CoT largely result from improved repository grounding rather than from the reasoning procedure itself.

In contrast, statically analyzable defects (e.g., runtime, concurrency, and type errors) are more effectively addressed through retrieval-augmented prompting, whereas structural interface mismatches often benefit from zero-shot prompting, which provides direct and sufficient guidance without the overhead of CoT reasoning. Consequently, the search for a universally optimal prompting strategy is inherently misguided: effective APR requires adaptive pipelines that dynamically select prompting strategies based on the interplay between bug characteristics and model architecture.

Despite recent progress, existing APR prompt engineering methods face several notable limitations that hinder real-world applicability. First, methodological idealization remains a primary concern; over 34% of current literature assumes 'perfect fault localization', thereby artificially inflating success rates and ignoring the noise (false positives) introduced by realistic localization tools. Second, a large portion of studies focuses on single-function bugs, overlooking the complexity of multi-file and cross-component defects. The field is also dominated by benchmarks like Defects4J and QuixBugs, which can lead to overfitting and poor generalization. Furthermore, evaluation inconsistencies—distinguishing plausible from correct patches and lacking quality metrics like maintainability—complicate fair comparisons. LLM non-determinism reduces reproducibility, and under-disclosed computational cost prevents practical decision regarding their use. Finally, context window constraints limit handling of repository-level bugs and attention dilution reduces performance as prompts grow longer (**Answer to RQ4**).

7. Threats to Validity

7.1. Internal validity

7.1.1. Selection and extraction bias

Threats to internal validity primarily stem from potential biases during literature search, study selection, and data extraction. Although our search covered major digital libraries and was supplemented by backward and forward snowballing, relevant studies may still have been missed due to coverage or indexing limitations. To minimize selection bias, two reviewers independently conducted a two-stage screening process (title/abstract review followed by full-text assessment) using predefined inclusion and exclusion criteria, achieving strong inter-rater agreement (Cohen's κ). Disagreements occurred mainly in borderline cases where studies utilized LLMs for multiple tasks, making the centrality of prompting to patch synthesis ambiguous. These cases were resolved through consensus discussions, though inclusion decisions ultimately rely on our operational definition of "prompt-centric" APR. Finally, data extraction presented interpretive challenges: several primary studies described prompting strategies informally or embedded them deep within broader repair frameworks, requiring us to infer specific prompt design details.

7.1.2. Classification and coding bias

Classification bias occurs when rigid frameworks misrepresent diverse literature. To mitigate this, we developed our taxonomy bottom-up, though early analyzed studies may have influenced initial category structures. To prevent subjective misclassification during extraction, we enforced a strict coding schema: when primary studies evaluated general benchmarks (e.g., Defects4J, QuixBugs, HumanEval-Fix) without reporting root-cause breakdowns, we categorized the Root-Cause Bug Type as 'Unreported' rather than imputing labels. This prevented post-hoc bias and ensured our taxonomy relies strictly on empirically reported primary data.

7.2. External Validity

Threats to external validity primarily stem from the characteristics of the surveyed primary studies. Most existing literature focuses on Python and Java using standard benchmarks (e.g., Defects4J, QuixBugs), which may limit direct generalizability to other programming languages, proprietary industrial codebases, or newer evaluation sets. To minimize selection bias, our multi-venue search imposed no restrictions on languages, benchmarks, or LLM architectures, synthesizing trends across literature published between January 2022 and June 2026. Consequently, our findings present a temporal snapshot of current capabilities rather than static claims. Crucially, our synthesis methodology and evaluation metrics establish an extensible framework that future research can adopt to evaluate emerging models and continuously assess the generalizability of our findings.

7.3. Construct Validity

The primary threat to construct validity lies in potential bias or subjective interpretation when mapping primary studies into our three-category taxonomy—particularly for hybrid strategies (e.g., Retrieval-Augmented Generation) that span multiple functional boundaries. To ensure our categories reflect meaningful operational constructs, we grounded the taxonomy directly in the sequential phases of an LLM generation pipeline: input formulation, reasoning generation, and output validation.

To ensure consistent classification across overlapping boundaries, we operationalized a strategy's *dominant trait* as the primary experimental variable isolated and evaluated in the target study. To validate these construct boundaries, two authors independently coded the complete corpus using these guidelines, achieving high inter-rater agreement (Cohen's $\kappa = 0.85$), with all initial discrepancies resolved through consensus.

8. Related Work

The convergence of prompt engineering and APR represents an active research area in software engineering. Despite extensive independent literature on APR techniques and prompt engineering, a systematic investigation into their intersection remains absent. This gap is notable; prompt design has become a primary determinant of success for LLM-based repair.

Table 6 positions this work against nine related surveys along thirty one criteria grouped into six families, spanning scope and methodology, prompt-design dimensions, APR pipeline coverage, model adaptation, evaluation practice, and reported outputs. Scoring at the criterion level rather than summarizing each survey narratively exposes where two surveys that nominally share a scope in fact diverge, and it makes the residual gap addressable rather than merely asserted. Three observations follow.

- First, coverage of the APR pipeline is broadly saturated: seven of the ten surveys treat patch generation as a first-class axis, so pipeline breadth no longer differentiates contributions in this space.
- Second, prompt design coverage remains far from exhaustive and highly fragmented: instruction design, context construction, reasoning scaffolds, and feedback loops are each addressed in full by at most two surveys. To our knowledge, no existing survey comprehensively covers more than two of these four dimensions simultaneously.
- Third, and most consequentially, the criterion *prompt-variant empirical comparison*, whether the survey reports how outcomes shift when the prompt is changed while model and benchmark are held fixed, is fully addressed by none of the nine.

Additionally, key criteria directly targeted by our research questions, test-suite quality/patch overfitting (RQ3) and context-window/cost constraints (RQ4)—are largely unaddressed in prior work. Only Zhang et al. [78] score a full mark on both, and Zhan et al. [77] score a full mark on the latter alone, in each case as a named item within a challenges or research-agenda section rather than as a controlled comparison of prompt variants; the remaining seven surveys leave both dimensions unaddressed or only partially so. The sparse region of Table 6 pinpoints the exact focus of this survey. By utilizing prompt engineering as an explicit analytical lens, this taxonomy provides a necessary extension to existing syntheses, bridging the gap between LLM configuration and repair efficacy.

We note that this specialization is deliberately purchased at a cost, visible in the fine-tuning and agentic repair rows of Table 6: those levers, which six and four prior surveys respectively treat as first-class dimensions, fall outside our scope and are scored accordingly. Our review is complementary to, not a replacement for, those syntheses.

Corpus size across related surveys reflects scope rather than evidence quality: broader surveys often cover hundreds of studies spanning diverse tasks or adaptation paradigms (e.g., Hou et al. [18]: 395 studies; Zhan et al. [77]: 91; Zhang et al. [78]: 189), where prompting is only one dimension among many. In contrast, our 53 studies specifically isolate prompt design with sufficient reporting detail to enable controlled comparisons across prompt variants, models, and benchmarks.

8.1. APR Surveys

Recent attempts to summarize progress in APR include two surveys that exemplify the current state of APR research. Huang et al. [20] review a filtered set of 67 works (drawn from an initial pool of 146 papers collected across top-10 APR venues from 2017–2023), classify techniques into four paradigms, and propose uniform evaluation criteria; while it highlights the critical opportunities LLMs bring to APR. While the authors acknowledge the potential of LLMs and briefly mention prompt engineering as a future opportunity, their work does not systematically investigate how prompt design influences repair performance. This asymmetry is visible in Table 6, where the survey attains full marks across the pipeline family (fault localization, patch generation, patch assessment) yet no mark at all across the prompt-design family.

Table 6: Comparison with nine related surveys. ● = fully covered / dedicated treatment; ◐ = partially covered / discussed as a secondary concern; ○ = not covered.

	Our study	Huang et al. [20]	Dikici and Bilgin [11]	Vatsal and Dubey [58]	Sasaki et al. [53]	Hou et al. [18]	Zubair et al. [84]	Zhan et al. [77]	Zhang et al. [78]	Yang et al. [71]
A. Scope and methodology										
Search Window	2022-2026	2017–2023	2013–2024	2022-2024	2021–2023	2017–2024	2021-2024	2021-2026	2020-2025	2022-2025
Dedicated APR focus	●	●	●	○	◐	◐	●	●	●	●
LLM-centric analysis	●	◐	◐	●	●	●	●	●	●	●
Prompting as the primary analytical lens	●	○	○	●	●	◐	○	○	◐	◐
Systematic selection protocol (SLR/PRISMA)	●	◐	●	◐	●	●	◐	●	●	◐
Initial records retrieved	2,013	146	10,738	N/A	158	218,765	431	1,491	7,253	1,581
Corpus size (primary studies)	**53**	67	41	44	28	395	41	91	189	62
Publication year	—	2024	2025	2024	2024	2024	2025	2026	2026	2025
Replication Package	●	○	○	○	○	●	○	○	●	●
B. Prompt-design dimensions										
Instruction design (role, zero-/few-shot)	●	○	○	●	●	◐	○	○	◐	◐
Context construction (code, bug report, tests)	●	◐	○	◐	◐	◐	○	◐	◐	●
Reasoning scaffolds (CoT, self-consistency)	●	○	○	●	◐	◐	○	○	◐	◐
Feedback loop / iterative repair	●	◐	○	○	○	○	◐	◐	◐	●
Prompt-variant comparison	●	○	◐	◐	○	○	○	○	○	◐
C. APR pipeline coverage										
Fault localization	◐	●	◐	○	○	◐	○	◐	◐	◐
Patch generation	●	●	●	○	◐	◐	●	●	●	●
Patch validation and ranking	◐	●	◐	○	○	◐	◐	◐	◐	●
Repair scenarios (vulnerability, multi-hunk, repo-level)	◐	◐	●	○	○	◐	●	◐	●	◐
D. Model and adaptation strategies										
LLM architecture families	●	◐	◐	○	○	●	●	●	●	●
Fine-tuning / PEFT	○	◐	◐	○	○	●	●	●	●	◐
Retrieval augmentation (RAG)	●	○	○	○	○	◐	○	◐	◐	●
Agentic / tool-augmented repair	◐	○	○	○	○	◐	○	○	◐	●
E. Evaluation practice										
Benchmark coverage	●	◐	●	●	○	●	◐	◐	●	●
Metric definition / harmonization	◐	●	●	◐	○	●	◐	◐	◐	●
Language coverage beyond Java	◐	◐	●	○	○	●	◐	◐	◐	◐
Data leakage / benchmark contamination	◐	○	○	○	○	◐	○	○	◐	◐
Cost and efficiency (tokens, latency)	●	○	○	◐	○	○	○	○	○	◐
F. Outputs and takeaways										
Novel taxonomy proposed	●	●	◐	●	●	●	◐	●	●	●
Empirical trade-off analysis	●	◐	●	◐	○	◐	◐	◐	◐	●
Actionable practitioner guidelines	●	◐	◐	◐	●	◐	◐	◐	◐	◐
Open challenges and research roadmap	●	●	●	◐	◐	●	●	●	●	●

Similarly, Dikici and Bilgin [11] analyze 41 APR tools across template-based, machine learning, and deep learning categories, offering a comprehensive review of methods and success rates. Although it touches upon learning-based approaches and evaluation metrics, its scope remains at the level of model categories and tool performance, leaving the granular engineering of prompts as a gap for specialized investigation. It is, together with, as discussed below, the more recent reviews of Zhan et al. [77] and Zhang et al. [78], among the surveys that treat language coverage beyond Java as an explicit dimension, a strength our review inherits only partially. Hou et al. [18], despite its comparatively larger corpus, discusses individual program-

ming languages only incidentally within specific tasks — e.g., TypeScript for type inference, JavaScript for program repair — rather than as a systematic coverage dimension, and is therefore not counted among these surveys.

8.2. Prompt Engineering Surveys

The literature on prompt engineering provides structural taxonomies that are relevant to software repair but often lack specificity for the APR domain. Vatsal and Dubey [58] surveys 39 prompting methods across 29 different NLP tasks, including code generation and logical reasoning (the paper's own task label, §3.2), categorizing techniques such as CoT and ToT. While it covers code-related tasks, its focus is general-purpose NLP. Table 6 makes the trade-off explicit: it is one of only three surveys with full coverage of reasoning scaffolds, but scores no mark on APR focus, patch generation, or repair scenarios, so its prompt-level depth does not transfer to repair-specific guidance.

More narrowly, Sasaki et al. [53] conducts a systematic literature review of prompt engineering in SE, identifying 21 patterns across five categories: learning approach, interaction-focused, task-specific, model optimization, and systematization and cataloging. It specifically mentions APR patterns like sequential prompting for bug identification. Its interaction-focused category is what earns it a partial mark on conversational repair, the only prompt-engineering survey in Table 6 to register on that criterion.[1] Yet, this work also reveals significant limitations. The reviewed studies predominantly treat APR as one task among many in a broader SE landscape, rarely examining prompt design choices specific to repair scenarios. Consistently, it records no mark on benchmark coverage or metric harmonization: its reusable patterns are proposed but not empirically graded against repair benchmarks.

8.3. LLM for Code Surveys

Several surveys examine LLMs applied to software engineering or code-related tasks but treat APR-specific prompting as a secondary concern. Hou et al. [18] analyzes 395 papers on LLMs in SE, noting optimization techniques like prompting but treating APR merely as one of 85 downstream tasks. Its corpus of 395 papers is the largest in Table 6, nearly twice the size of the next-largest (Zhang et al.'s 189 papers [78]), yet this breadth converts to partial rather than full marks on every APR pipeline criterion, illustrating that corpus size and depth on a given axis are largely independent.

Focusing specifically on APR, Zubair et al. [84] survey LLMs for APR, analyzing 41 studies with a distinct focus on fine-tuning techniques, datasets, and evaluation metrics. It is accordingly the one survey scoring full marks on benchmark coverage, metric harmonization, and repair scenarios, while treating prompting only as a single undifferentiated entry within its capability and technique inventories rather than as a decomposed design space, a profile that positions it as a close but not total complement to our own inverted profile.

Zhan et al. [77] contribute a systematic review of Code Pre-trained Models, a model class distinct from general-purpose LLMs, spanning 91 studies published through April 2026. Rather than isolating prompting, they organize the field around five integration paradigms, fine-tuning, in-context learning, feedback-driven repair, autonomous agents, and patch assessment, with in-context learning itself split only into zero-shot and few-shot categories. Their explicit aim is to characterize how CodePTM architecture and pre-training objectives shape integration into the APR pipeline, not to dissect prompt design. Three of those five paradigms map to full marks in Table 6 (feedback loop, agentic repair, patch assessment), but the fourth collapses the whole of prompt design onto a single zero-shot/few-shot axis, leaving reasoning scaffolds unaddressed; their CodePTM focus is also why we score them as only partially LLM-centric. Their evaluation apparatus is nonetheless comparatively rigorous for a 2026 review: dedicated subsections quantify benchmark and metric usage, track programming-language coverage, and name inference cost and benchmark contamination as explicit open challenges with concrete mitigation strategies, so the survey earns full or partial marks across most of the evaluation-practice and outputs families even though its treatment of prompt design remains narrow.

Taken together, these surveys establish the benchmark landscape and confirm the empirical dominance of LLMs in APR; they systematically neglect the internal structure and comparative effectiveness of prompting strategies—the central contribution of this work.

8.4. Competing Surveys

The most closely related work is Zhang et al. [78], who conducted an SLR dedicated exclusively to LLM-based APR, analyzing 189 papers published between 2020 and September 2025. Their review identifies four high-level adaptation strategies: fine-tuning, zero-shot prompting, few-shot prompting, and agent-based approaches, providing an indispensable broad-scope mapping of the field. While their review provides valuable breadth on which LLMs and repair scenarios have been explored, its treatment of prompting itself is coarse: zero-shot and conversational repair are merged into a single paper bucket despite representing mechanistically distinct strategies.

Table 6 quantifies exactly this: across the five prompt-design criteria they attain one full mark (context construction, via the dedicated six-way input-form taxonomy that organizes their integration-perspective analysis), three partial marks, and no mark on reasoning scaffolds or prompt-variant comparison, against full marks on patch generation, patch validation and assessment, repair scenarios, and agentic repair. Their evaluation apparatus is thorough in the same asymmetric way: theirs is the only related survey to register full marks simultaneously on benchmark coverage, test-suite quality and patch overfitting, and context-window

[1] Worth noting: the cited interaction-focused patterns (PB1a, PB1b) concern general conversational programming assistants and object-model synthesis rather than repair specifically; the paper's APR-specific patterns (PA2a zero-shot APR/vulnerability repair, PC1a sequential bug-fix prompting, PC1b hardware-security debugging) instead fall under its learning-approach and task-specific categories.

and cost constraints, reflecting dedicated discussions of patch-correctness assessment, data-leakage mitigation, and inference cost organized as an explicit research agenda with actionable recommendations. Their taxonomy resolves the model, scenario, and evaluation axes finely and the prompt axis coarsely; ours does the reverse. Further, several of these hybrid or fine-tuning-adjacent techniques do not meet our reproducibility criteria for prompt-driven synthesis and are consequently outside our scope.

Our review differs across three dimensions: (i) scope, isolating prompting while treating fine-tuning and agentic architectures as orthogonal levers; (ii) granularity, introducing a purpose-built taxonomy of three families and fifteen subcategories; and (iii) analytical aim, synthesizing empirical evidence on prompt effectiveness across benchmark complexity, test-suite quality, and context-window limitations (RQ2–RQ4). These three aims correspond directly to the evaluation-practice criteria of Table 6; Zhang et al.'s own review is, in fact, the only one of the nine related surveys to score a full mark on all three simultaneously, but as a named item on their future-work agenda rather than as an empirical account of how prompt variation performs under those constraints, which is the gap our synthesis is designed to close. We provide depth on the specific design axis — prompt engineering — that increasingly dominates practice.

Similarly, Yang et al. [71] propose a unified taxonomy of 62 LLM-based repair systems based on control authority and parameter adaptation, distinguishing between fine-tuning, prompting, procedural, and agentic paradigms. While this high-level classification is valuable, it subdivides single-turn prompting only by exemplar count and augmentation source (zero-shot, few-shot, +RAG, +AAG), without treating instruction phrasing or reasoning-scaffold strategies as a first-class dimension — techniques such as chain-of-thought repair are instead folded into their Procedural paradigm rather than analyzed as prompt-design choices. Of the nine related surveys it is the closest to ours on the criteria we prioritize, registering partial marks on prompt-variant comparison, test-suite quality, and context-window constraints, the only survey to register on all three. The distinction is therefore one of resolution rather than of subject: their prompting paradigm is a single cell in a four-paradigm taxonomy, whereas it is the entire object of our fifteen-subcategory decomposition. Our review addresses this shared gap by targeting the prompting paradigm exclusively, offering a granular analysis of prompt strategies for APR.

9. Conclusion

We presented a taxonomy of LLM-based APR prompting strategies based on 53 primary studies published between January 2022 and June 2026. By analyzing prompting strategies across context-awareness, reasoning, and quality-assurance dimensions, we show that prompt engineering has become a key factor in repair effectiveness. Our findings reveal a shift from static zero-shot approaches toward more dynamic workflows, such as RAG and CoT, while highlighting that strategy effectiveness depends on the interaction between prompting design, model capabilities, and defect characteristics. In particular, reasoning-oriented strategies benefit complex logical and semantic repairs, whereas lightweight prompting remains effective for simpler structural defects.

However, current evidence is limited by evaluation gaps, including benchmark bias, test-suite dependency, and simplifying assumptions such as perfect fault localization and single-file repair scopes. These limitations hinder conclusions about the generalizability of prompt-based APR to real-world software systems. Future research should therefore focus on repository-scale repair, cost-aware prompt selection, and evaluation protocols that incorporate semantic correctness and human-centered quality criteria beyond test-based plausibility.

Overall, advancing prompt-based APR requires moving from isolated benchmark improvements toward adaptive methodologies that dynamically select and combine prompting strategies according to bug characteristics, program context, and model capabilities, while ensuring realistic and cost-effective repair processes.

Declaration on the Use of Generative AI

During the preparation of this work, the authors used generative AI tools (Claude and Google Gemini) solely to assist with language editing and readability. All AI-generated suggestions were thoroughly reviewed, edited, and validated by the authors, who take full responsibility for the accuracy of the manuscript and the integrity of all reported results.

References

[1] Abdinabiev, A.S., Jung, E., Lee, B., 2026. Utilizing dynamic context and static analysis for agent-based automated program repair. IEEE Access .

[2] Ahmed, T., Devanbu, P., 2023. Better patching using llm prompting, via self-consistency, in: 2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE), IEEE. pp. 1742–1746.

[3] Bajpai, Y., Chopra, B., Biyani, P., Aslan, C., Coleman, D., Gulwani, S., Parnin, C., Radhakrishna, A., Soares, G., 2024. Let's fix this together: Conversational debugging with github copilot, in: 2024 IEEE Symposium on Visual Languages and Human-Centric Computing (VL/HCC), IEEE. pp. 1–12.

[4] Bouzenia, I., Devanbu, P., Pradel, M., 2025. Repairagent: An autonomous, llm-based agent for program repair, in: 2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE), pp. 2188–2200.

[5] Britton, T., Jeng, L., Carver, G., Cheak, P., Katzenellenbogen, T., 2013. Reversible debugging software. Judge Bus. School, Univ. Cambridge, Cambridge, UK, Tech. Rep 229, 2013.

[6] Cai, X., Jiang, L., 2025. Adapting knowledge prompt tuning for enhanced automated program repair, in: 2025 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER), IEEE. pp. 360–371.

[7] Cao, J., Li, M., Wen, M., Cheung, S.c., 2025. A study on prompt design, advantages and limitations of chatgpt for deep learning program repair. Automated Software Engineering 32, 1–29.

[8] Chen, M., Tworek, J., Jun, H., Yuan, Q., Pinto, H.P.D.O., Kaplan, J., Edwards, H., Burda, Y., Joseph, N., Brockman, G., et al., 2021. Evaluating large language models trained on code. arXiv preprint arXiv:2107.03374 .

[9] Chen, Y., Wu, J., Ling, X., Li, C., Rui, Z., Luo, T., Wu, Y., 2024. When large language models confront repository-level automatic program repair: How well they done?, in: Proceedings of the 2024 IEEE/ACM 46th International Conference on Software Engineering: Companion Proceedings, pp. 459–471.

[10] Darwiyanto, E., Gusnaen, R., Nurtantyana, R., 2025. A comparative study of large language models with chain-of-thought prompting for automated program repair. IAES International Journal of Artificial Intelligence (IJ-AI) 14, 4579.

[11] Dikici, S., Bilgin, T.T., 2025. Advancements in automated program repair: a comprehensive review. Knowledge and Information Systems , 1–47.

[12] Do Viet, T., Markov, K., 2023. Using large language models for bug localization and fixing, in: 2023 12th International Conference on Awareness Science and Technology (iCAST), IEEE. pp. 192–197.

[13] Ehsani, R., Parra, E., Haiduc, S., Chatterjee, P., 2025. Hierarchical knowledge injection for improving llm-based program repair, in: 2025 40th IEEE/ACM International Conference on Automated Software Engineering (ASE), IEEE. p. 1440–1452.

[14] Fan, Z., Gao, X., Mirchev, M., Roychoudhury, A., Tan, S.H., 2023. Automated repair of programs from large language models, in: 45th IEEE/ACM International Conference on Software Engineering (ICSE), IEEE. pp. 1469–1481.

[15] Feng, Z., Guo, D., Tang, D., Duan, N., Feng, X., Gong, M., Shou, L., Qin, B., Liu, T., Jiang, D., et al., 2020. Codebert: A pre-trained model for programming and natural languages, in: Findings of the association for computational linguistics: EMNLP 2020, pp. 1536–1547.

[16] Hao, S., Shi, X., Liu, H., 2024. Retyper: Integrated retrieval-based automatic program repair for python type errors, in: 2024 IEEE International Conference on Software Maintenance and Evolution (ICSME), IEEE. pp. 199–210.

[17] Haque, M., Babkin, P., Farmahinifarahani, F., Veloso, M., 2025. Towards effectively leveraging execution traces for program repair with code llms, in: Proceedings of the 4th International Workshop on Knowledge-Augmented Methods for Natural Language Processing, Association for Computational Linguistics. pp. 160–179.

[18] Hou, X., Zhao, Y., Liu, Y., Yang, Z., Wang, K., Li, L., Luo, X., Lo, D., Grundy, J., Wang, H., 2024. Large language models for software engineering: A systematic literature review. ACM Transactions on Software Engineering and Methodology 33, 1–79.

[19] Hu, H., Shang, Y., Xu, G., He, C., Zhang, Q., 2025. Can gpt-o1 kill all bugs? an evaluation of gpt-family llms on quixbugs, in: 2025 IEEE/ACM International Workshop on Automated Program Repair (APR), IEEE. pp. 11–18.

[20] Huang, K., Xu, Z., Yang, S., Sun, H., Li, X., Yan, Z., Zhang, Y., 2024. Evolving paradigms in automated program repair: Taxonomy, challenges, and opportunities. ACM Computing Surveys 57, 1–43.

[21] Hurst, A., Lerer, A., Goucher, A.P., Perelman, A., Ramesh, A., Clark, A., Ostrow, A., Welihinda, A., Hayes, A., Radford, A., et al., 2024. Gpt-4o system card. arXiv preprint arXiv:2410.21276 .

[22] Jiang, N., Liu, K., Lutellier, T., Tan, L., 2023. Impact of code language models on automated program repair, in: 2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE), IEEE. pp. 1430–1442.

[23] Jin, H., Sun, Z., Chen, H., 2024. Rgd: Multi-llm based agent debugger via refinement and generation guidance, in: 2024 IEEE International Conference on Agents (ICA), IEEE. pp. 136–141.

[24] Jin, M., Shahriar, S., Tufano, M., Shi, X., Lu, S., Sundaresan, N., Svyatkovskiy, A., 2023. Inferfix: End-to-end program repair with llms, in: Proceedings of the 31st ACM joint european software engineering conference and symposium on the foundations of software engineering, pp. 1646–1656.

[25] Joshi, H., Sanchez, J.C., Gulwani, S., Le, V., Verbruggen, G., Radiček, I., 2023. Repair is nearly generation: Multilingual program repair with llms, in: Proceedings of the AAAI Conference on Artificial Intelligence, pp. 5131–5140.

[26] Kang, S., Chen, B., Yoo, S., Lou, J.G., 2025. Explainable automated debugging via large language model-driven scientific debugging. Empirical Software Engineering 30, 45.

[27] Kitchenham, B., Charters, S., et al., 2007. Guidelines for performing systematic literature reviews in software engineering .

[28] Kong, J., Xie, X., Cheng, M., Liu, S., Du, X., Guo, Q., 2025. Contrastrepair: Enhancing conversation-based automated program repair via contrastive test case pairs. ACM Trans. Softw. Eng. Methodol. .

[29] Lee, C., Xia, C.S., Yang, L., Huang, J.t., Zhu, Z., Zhang, L., Lyu, M.R., 2025. Unidebugger: Hierarchical multi-agent framework for unified software debugging, in: Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing, pp. 18248–18277.

[30] Lester, B., Al-Rfou, R., Constant, N., 2021. The power of scale for parameter-efficient prompt tuning, in: Proceedings of the 2021 conference on empirical methods in natural language processing, pp. 3045–3059.

[31] Li, K., Wu, S., Tang, X., Yao, C., Bu, S., Xu, Q., Chen, G., 2026. Debugging engine enhanced by prior knowledge: Can we teach llm how to debug? Proceedings of the ACM on Software Engineering (FSE) 3, 1823–1843.

[32] Li, Y., Cai, M., Chen, J., Xu, Y., Huang, L., Li, J., 2025. Context-aware prompting for llm-based program repair. Automated Software Engineering 32, 42.

[33] Li, Y., Guo, J., 2024. Research on program automatic repair method combining context optimization strategy and large language models, in: 2024 4th International Symposium on Computer Technology and Information Science (ISCTIS), IEEE. pp. 26–34.

[34] Liu, L., Li, M., Ren, J., 2026. Impact of different buggy code information on large language model-based automated program repair, in: 2026 6th International Symposium on Computer Technology and Information Science (ISCTIS), IEEE. pp. 319–328.

[35] Liu, Q., Bu, X., Yan, Z., Li, R., 2025a. T3: Multi-level tree-based automatic program repair with large language models, in: 2025 International Joint Conference on Neural Networks (IJCNN), IEEE. pp. 1–8.

[36] Liu, Z., Du, X., Liu, H., 2025b. Reapr: Automatic program repair via retrieval-augmented large language models: Z. liu, x. du, h. liu. Software Quality Journal 33, 30.

[37] Long, D.X., Yen, D.N., Luu, A.T., Kawaguchi, K., Kan, M.Y., Chen, N.F., 2024. Multi-expert prompting improves reliability, safety, and usefulness of large language models, in: Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing, Association for Computational Linguistics. pp. 20370–20401.

[38] Madaan, A., Tandon, N., Gupta, P., Hallinan, S., Gao, L., Wiegreffe, S., Alon, U., Dziri, N., Prabhumoye, S., Yang, Y., et al., 2023. Self-refine: Iterative refinement with self-feedback. Advances in Neural Information Processing Systems 36, 46534–46594.

[39] Mansur, E., Chen, J., Raza, M.A., Wardat, M., 2024. Ragfix: Enhancing llm code repair using rag and stack overflow postshao2024retyper, in: 2024 IEEE International Conference on Big Data (BigData), IEEE. pp. 7491–7496.

[40] Martinez, M., Monperrus, M., 2019. Astor: Exploring the design space of generate-and-validate program repair beyond genprog. Journal of Systems and Software 151, 65–80.

[41] Mu, F., Wang, J., Shi, L., Wang, S., Li, S., Wang, Q., 2026. Experepair: Dual-memory enhanced llm-based repository-level program repair. Proceedings of the ACM on Software Engineering (issue FSE) 3, 3954–3976.

[42] Nashid, N., Sintaha, M., Mesbah, A., 2023. Retrieval-based prompt selection for code-related few-shot learning, in: 2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE), IEEE. pp. 2450–2462.

[43] Nguyen, T.T., Zhang, X.Y., Arcaini, P., Ishikawa, F., Vo, H.D., 2025. Automated program repair for variability bugs in software product line systems. Journal of Systems and Software 221, 112152.

[44] Omari, S., Basnet, K., Wardat, M., 2024. Investigating large language models capabilities for automatic code repair in python. Cluster Computing 27, 10717–10731.

[45] Ouyang, L., Wu, J., Jiang, X., Almeida, D., Wainwright, C., Mishkin, P., Zhang, C., Agarwal, S., Slama, K., Ray, A., et al., 2022. Training language models to follow instructions with human feedback. Advances in neural information processing systems 35, 27730–27744.

[46] Ouyang, S., Zhang, J.M., Sun, Z., Penuela, A.M., 2025. Knowledge-enhanced program repair for data science code, in: 2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE), IEEE. pp. 898–910.

[47] Page, M.J., McKenzie, J.E., Bossuyt, P.M., Boutron, I., Hoffmann, T.C., Mulrow, C.D., Shamseer, L., Tetzlaff, J.M., Akl, E.A., Brennan, S.E., et al., 2021. The prisma 2020 statement: an updated guideline for reporting systematic reviews. bmj 372.

[48] Parasaram, N., Yan, H., Yang, B., Flahy, Z., Qudsi, A., Ziaber, D., Barr, E.T., Mechtaev, S., 2025. The fact selection problem in llm-based program repair, in: 2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE), pp. 2574–2586.

[49] Prenner, J.A., Babii, H., Robbes, R., 2022. Can openai's codex fix bugs? an evaluation on quixbugs, in: Proceedings of the Third International Workshop on Automated Program Repair (APR), pp. 69–75.

[50] Qu, X., Zuo, F., Li, X., Rhee, J., 2024. Context matters: Investigating its impact on chatgpt's bug fixing performance, in: 2024 IEEE/ACIS 22nd International Conference on Software Engineering Research, Management and Applications (SERA), IEEE. pp. 17–23.

[51] Rondon, P., Wei, R., Cambronero, J., Cito, J., Sun, A., Sanyam, S., Tufano, M., Chandra, S., 2025. Evaluating agent-based program repair at google, in: 2025 IEEE/ACM 47th International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP), pp. 365–376.

[52] Ruan, H., Zhang, Y., Roychoudhury, A., 2025. Specrover: Code intent extraction via llms, in: Proceedings of the IEEE/ACM 47th International Conference on Software Engineering, IEEE. p. 963–974.

[53] Sasaki, Y., Washizaki, H., Li, J., Sander, D., Yoshioka, N., Fukazawa, Y., 2024. Systematic literature review of prompt engineering patterns in software engineering, in: 2024 IEEE 48th Annual Computers, Software, and Applications Conference (COMPSAC), IEEE. pp. 670–675.

[54] Shanahan, M., McDonell, K., Reynolds, L., 2023. Role play with large language models. Nature 623, 493–498.

[55] Shi, Y., Bangash, A.A., Fallahzadeh, E., Adams, B., Hassan, A.E., 2026. Hafix: History-augmented large language models for bug fixing. Empirical Software Engineering 31, 104.

[56] Sobania, D., Briesch, M., Hanna, C., Petke, J., 2023. An analysis of the automatic bug fixing performance of chatgpt, in: 2023 IEEE/ACM International Workshop on Automated Program Repair (APR), IEEE. pp. 23–30.

[57] Tang, H., Hu, K., Zhou, J., Zhong, S.C., Zheng, W.L., Si, X., Ellis, K., 2024. Code repair with llms gives an exploration-exploitation tradeoff. Advances in Neural Information Processing Systems 37, 117954–117996.

[58] Vatsal, S., Dubey, H., 2024. A survey of prompt engineering methods in large language models for different nlp tasks. arXiv preprint arXiv:2407.12994 .

[59] Wang, X., Wei, J., Schuurmans, D., Le, Q., Chi, E., Narang, S., Chowdhery, A., Zhou, D., 2023. Self-consistency improves chain of thought reasoning in language models, in: International Conference on Learning Representations (ICLR).

[60] Wang, Y., Wang, W., Joty, S., Hoi, S.C., 2021. Codet5: Identifier-aware unified pre-trained encoder-decoder models for code understanding and generation, in: Proceedings of the 2021 conference on empirical methods in natural language processing, pp. 8696–8708.

[61] Wei, J., Bosma, M., Zhao, V.Y., Guu, K., Yu, A.W., Lester, B., Du, N., Dai, A.M., Le, Q.V., 2022a. Finetuned language models are zero-shot learners, in: The Tenth International Conference on Learning Representations (ICLR).

[62] Wei, J., Wang, X., Schuurmans, D., Bosma, M., Xia, F., Chi, E., Le, Q.V., Zhou, D., et al., 2022b. Chain-of-thought prompting elicits reasoning in large language models. Advances in neural information processing systems 35, 24824–24837.

[63] Wohlin, C., 2014. Guidelines for snowballing in systematic literature studies and a replication in software engineering, in: Proceedings of the 18th international conference on evaluation and assessment in software engineering, pp. 1–10.

[64] Wuisang, M.C., Kurniawan, M., Santosa, K.A.W., Gunawan, A.A.S., Saputra, K.E., 2023. An evaluation of the effectiveness of openai's chatgpt for automated python program bug fixing using quixbugs, in: 2023 International Seminar on Application for Technology of Information and Communication (iSemantic), IEEE. pp. 295–300.

[65] Xia, C.S., Deng, Y., Dunn, S., Zhang, L., 2025. Demystifying llm-based software engineering agents. Proceedings of the ACM on Software Engineering (FSE) 2, 801–824.

[66] Xia, C.S., Wei, Y., Zhang, L., 2023. Automated program repair in the era of large pre-trained language models, in: 2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE), IEEE. pp. 1482–1494.

[67] Xia, C.S., Zhang, L., 2022. Less training, more repairing please: revisiting automated program repair via zero-shot learning, in: Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering, pp. 959–971.

[68] Xia, C.S., Zhang, L., 2024. Automated program repair via conversation: Fixing 162 out of 337 bugs for $0.42 each using chatgpt, in: Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis, pp. 819–831.

[69] Xu, B., Yang, A., Lin, J., Wang, Q., Zhou, C., Zhang, Y., Mao, Z., 2023. Expertprompting: Instructing large language models to be distinguished experts. arXiv preprint arXiv:2305.14688 .

[70] Xu, J., Fu, Y., Tan, S.H., He, P., 2025. Aligning the objective of llm-based program repair, in: 2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE), pp. 2548–2560.

[71] Yang, B., Cai, Z., Liu, F., Le, B., Zhang, L., Bissyandé, T.F., Liu, Y., Tian, H., 2025. A survey of llm-based automated program repair: Taxonomies, design paradigms, and applications. arXiv preprint arXiv:2506.23749 .

[72] Yang, B., Tian, H., Pian, W., Yu, H., Wang, H., Klein, J., Bissyandé, T.F., Jin, S., 2024. Cref: An llm-based conversational software repair framework for programming tutors, in: Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis, pp. 882–894.

[73] Yao, S., Yu, D., Zhao, J., Shafran, I., Griffiths, T., Cao, Y., Narasimhan, K., 2023a. Tree of thoughts: Deliberate problem solving with large language models. Advances in neural information processing systems 36, 11809–11822.

[74] Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., Cao, Y., 2023b. React: Synergizing reasoning and acting in language models, in: International Conference on Learning Representations (ICLR).

[75] Ye, H., Yang, A.Z., Hu, C., Wang, Y., Zhang, T., Le Goues, C., 2025. Adverintent-agent: Adversarial reasoning for repair based on inferred program intent. Proceedings of the ACM on Software Engineering (ISSTA) 2, 1398–1420.

[76] Yin, X., Ni, C., Wang, S., Li, Z., Zeng, L., Yang, X., 2024. Thinkrepair: Self-directed automated program repair, in: Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis, pp. 1274–1286.

[77] Zhan, S., Wang, X., Wei, D., Xiang, X., Lin, Y., Wu, H., Lu, J., 2026. Unveiling the power of code pre-trained models in automated program repair: A systematic review. Journal of Systems and Software , 113015.

[78] Zhang, Q., Fang, C., Xie, Y., Ma, Y., Sun, W., Yang, Y., Chen, Z., 2026a. A systematic literature review on large language models for automated program repair. ACM Trans. Softw. Eng. Methodol. .

[79] Zhang, R., Zhang, S., Xie, L., 2026b. Towards defect-type-aware adaptive program repair: A stage-wise approach with large language models. Software Quality Journal 34, 14.

[80] Zhang, Y., Jin, Z., Xing, Y., Li, G., Liu, F., Zhu, J., Dou, W., Wei, J., 2025. Patch: Empowering large language model with programmer-intent guidance and collaborative-behavior simulation for automatic bug fixing. ACM Transactions on Software Engineering and Methodology .

[81] Zhang, Y., Ruan, H., Fan, Z., Roychoudhury, A., 2024. Autocoderover: Autonomous program improvement, in: Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis, pp. 1592–1604.

[82] Zhao, J., Yang, D., Zhang, L., Lian, X., Yang, Z., Liu, F., 2024. Enhancing automated program repair with solution design, in: Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering, pp. 1706–1718.

[83] Zhong, L., Wang, Z., Shang, J., 2024. Debug like a human: A large language model debugger via verifying runtime execution step by step, in: Findings of the Association for Computational Linguistics: ACL 2024, Association for Computational Linguistics. pp. 851–870.

[84] Zubair, F., Al-Hitmi, M., Catal, C., 2025. The use of large language models for program repair. Computer Standards & Interfaces 93, 103951.

Appendix A. Query Construction and Database Adaptations

Table A.7: Database-specific Query Adaptations.

Database	Field scope	Boolean query string
IEEE Xplore	Document Title; Abstract (Command Search)	(("Document Title":"large language model" OR "Document Title":LLM OR "Document Title":GPT OR "Document Title":CodeLlama OR "Document Title":StarCoder OR "Document Title":Codex OR "Document Title":"transformer model" OR "Document Title":PLM OR "Document Title":CodeT5 OR "Document Title":pretrained OR "Document Title":"pre-trained" OR "Abstract":"large language model" OR "Abstract":LLM OR "Abstract":GPT OR "Abstract":CodeLlama OR "Abstract":StarCoder OR "Abstract":Codex OR "Abstract":"transformer model" OR "Abstract":PLM OR "Abstract":CodeT5 OR "Abstract":pretrained OR "Abstract":"pre-trained" OR "Document Title":"prompt engineering" OR "Document Title":prompting OR "Document Title":"few-shot" OR "Document Title":"zero-shot" OR "Document Title":CoT OR "Document Title":"chain-of-thought" OR "Document Title":ToT OR "Document Title":React OR "Document Title":RAG OR "Document Title":"instruction tuning" OR "Abstract":"prompt engineering" OR "Abstract":prompting OR "Abstract":"few-shot" OR "Abstract":"zero-shot" OR "Abstract":CoT OR "Abstract":"chain-of-thought" OR "Abstract":ToT OR "Abstract":React OR "Abstract":RAG OR "Abstract":"instruction tuning") AND ("Document Title":"program repair" OR "Document Title":"automatic repair" OR "Document Title":"bug fix" OR "Document Title":"patch generation" OR "Document Title":"code repair" OR "Document Title":"defect repair" OR "Document Title":APR OR "Document Title":"automated program repair" OR "Document Title":"bug repair" OR "Abstract":"program repair" OR "Abstract":"automatic repair" OR "Abstract":"bug fix" OR "Abstract":"patch generation" OR "Abstract":"code repair" OR "Abstract":"defect repair" OR "Abstract":APR OR "Abstract":"automated program repair" OR "Abstract":"bug repair"))
Scopus	TITLE-ABS-KEY (combined title, abstract, keywords)	(TITLE-ABS-KEY("large language model" OR LLM OR GPT OR CodeLlama OR StarCoder OR Codex OR "transformer model" OR PLM OR CodeT5 OR pretrained OR "pre-trained") OR TITLE-ABS-KEY("prompt engineering" OR prompting OR "few-shot" OR "zero-shot" OR CoT OR "chain-of-thought" OR ToT OR React OR RAG OR "instruction tuning")) AND TITLE-ABS-KEY("program repair" OR "automatic repair" OR "bug fix" OR "patch generation" OR "code repair" OR "defect repair" OR APR OR "automated program repair" OR "bug repair")
ACM Digital Library	Title; Abstract	((Title:("large language model" OR llm OR gpt OR codellama OR starcoder OR codex OR "transformer model" OR plm OR codet5 OR pretrained OR "pre-trained") OR Abstract:("large language model" OR llm OR gpt OR codellama OR starcoder OR codex OR "transformer model" OR plm OR codet5 OR pretrained OR "pre-trained")) OR (Title:("prompt engineering" OR prompting OR "few-shot" OR "zero-shot" OR cot OR "chain-of-thought" OR tot OR react OR rag OR "instruction tuning") OR Abstract:("prompt engineering" OR prompting OR "few-shot" OR "zero-shot" OR cot OR "chain-of-thought" OR tot OR react OR rag OR "instruction tuning"))) AND (Title:("program repair" OR "automatic repair" OR "bug fix" OR "patch generation" OR "code repair" OR "defect repair" OR apr OR "automated program repair" OR "bug repair") OR Abstract:("program repair" OR "automatic repair" OR "bug fix" OR "patch generation" OR "code repair" OR "defect repair" OR apr OR "automated program repair" OR "bug repair"))
Google Scholar	Full text (no field restriction)	(("large language model" OR LLM OR GPT OR CodeLlama OR StarCoder OR Codex OR "transformer model" OR PLM OR CodeT5 OR pretrained OR "pre-trained") OR ("prompt engineering" OR prompting OR "few-shot" OR "zero-shot" OR CoT OR "chain-of-thought" OR ToT OR React OR RAG OR "instruction tuning")) AND ("program repair" OR "automatic repair" OR "bug fix" OR "patch generation" OR "code repair" OR "defect repair" OR APR OR "automated program repair" OR "bug repair")

Appendix B. Structure and Context-based

This appendix presents the complete per-study extraction for approaches classified under Structure and Context-Based prompting strategies, corresponding to the techniques discussed in Section 3.1. These studies share a common focus on enriching the LLM input through prompt structure, role assignment, or contextual information such as execution traces, test cases, and bug descriptions. Each row reports the prompting strategy, LLM backbone, evaluation benchmark(s), primary metric, key results, underlying assumptions (e.g., perfect fault localization or single-function scope), and target programming language(s). When multiple LLM backbones are evaluated, the results column corresponds to the model marked with an asterisk (*).

Table B.8: Summary of studies categorized by structure and context

Approach	Prompting Strategy	LLM backbone	Eval. Bench.	Key Metric	Results	Key Assumptions	Prog. Lang.
Prenner et al. (2022) [49]	Zero-shot	Codex	QuixBugs	#Plaussible patches	23 Py, 14 Java	Single-function	Java, Python
AlphaRepair (2022) [67]	Zero-shot	CodeBERT	Defects4J v1.2, Defects4J v2.0, QuixBugs	#Correct patches	D4J v1.2: 74, D4J v2.0: 36, QBs: 28 Java, QBs: 27 Py	Perfect FL, single-line	Java, Python
Fan et al. (2023) [14]	Zero-shot	Codex	LMDefects	#Correct patches	46	-	Java
Wuisang et al. (2023) [64]	Zero-shot	GPT-3.5	QuixBugs	Pass@4	30	Single-function	Python
Omari et al. (2024) [44]	Zero-shot	GPT-3.5-turbo	QuixBugs, Textbook	#Plaussible patches	QBs: 19, TB : 122	Single-function	Python
Hu et al. (2025) [19]	Zero-shot	**O1-preview***, O1-mini, GPT-4o, ChatGPT	QuixBugs	Repair Success Rate	40	Single-function	Python
Xia et al. (2023) [66]	Few-shot	**Codex***, GPT-Neo, GPT-J, GPT-NeoX, CodeT5, INCODER	Defects4J v1.2, Defects4J v2.0, QuixBugs, ManyBugs	#Correct patches	D4J v1.2: 63, D4J v2.0: 49, QBs: 32 Java, QBs: 37 Py, MBs: 7	Perfect FL, single-hunk	Java, Python, C
Cedar (2023) [42]	Few-shot	Codex	TFIX	Exact Match Accuracy (%)	51.72	-	JavaScript
Do et al. (2023) [12]	Few-shot	**GPT-3.5-turbo***, Galpaca-30b	QuixBugs	Exact Match Accuracy (%), Execution Accuracy (%)	62.5 (EMA), 87.5 (EA)	Single-function	Python
D4C (2025) [70]	One-shot	**GPT-4***, Mixtral-MoE	Defects4J v1.2, Defects4J v2.0, DebugBench	#Correct patches	D4J v1.2: 84, D4J v2.0: 96, DB: 177 C++, DB: 163 Java, DB: 161 Py	Single-function	Java, C++, Python
LDB (2024) [83]	Contextual	**GPT-3.5***, CodeLlama, StarCoder	HumanEval, MBPP, TransCoder	Pass@1 (%)	HE: 82.9, MBPP: 76, TC: 87.7	-	Python
Qu et al. (2024) [50]	Contextual	GPT-4	FIXEVAL	Execution Accuracy (%)	83.3	-	Python
Haque et al. (2025) [17]	Contextual	**GPT-4***, GPT-3.5	Refactory, HumanEval-Java, RunBugRun	Correct Program Accuracy (%)	RF: 88.4, HE-J: 71.3, RBR: 62.9	Single-function	Java, Python
Cao et al. (2025) [7]	Contextual	GPT-3.5-Turbo	DeepFD	#Correct patches	43	Perfect FL	Python
Liu et al. (2026) [34]	Contextual	Qwen3-Coder-480B*, Qwen2.5-Coder 7B/14B, CodeLlama 7B/13B, DeepseekV3.2	QuixBugs	Repair Success Rate (%)	97.5	Perfect FL, Single-function	Python
HAFix (2026) [55]	Contextual	DeepSeek-Coder-V2-Lite-16B*, CodeLlama-7B, DeepSeek-Coder-6.7B	Defects4J (116 bugs), BugsInPy (51 bugs)	Pass@1 (%)	Defects4J: 39.57, BugsInPy: 29.61	Perfect FL, Single-function	Java, Python

Note: When multiple LLM backbones are listed, the Results column reports the values corresponding to the model annotated with *.

Appendix C. Cognitive and Reasoning

This appendix presents the per-study extraction for approaches under Cognitive and Reasoning Enhancement strategies, as discussed in Section 3.2. Studies focus on improving LLMs' logical reasoning via Chain-of-Thought reasoning, Tree-of-Thought exploration, retrieval-augmented generation, self-refinement loops, and ReAct-based workflows. Each row lists the prompting strategy, LLM backbone, evaluation benchmark(s), primary metric, key results, assumptions, and target programming language(s). When multiple LLM backbones are evaluated, results correspond to the model marked with an asterisk (*).

Table C.9: Summary of studies categorized by cognitive and reasoning enhancement

Approach	Prompting Strategy	LLM backbone	Eval. Bench.	Key Metric	Results	Key Assumptions	Prog. Lang.
ThinkRepair (2024) [76]	CoT	**GPT-3.5-turbo***, CodeLlama, DeepSeek-Coder, StarCoder	Defects4J v1.2, Defects4J v2.0, QuixBugs	#Correct patches	D4J v1.2: 98, D4J v2.0: 107, QBs: 39 Java, QBs: 40 Py	Perfect FL, Single-function	Java, Python
Maniple (2025) [48]	CoT	**GPT-3.5-Turbo***, Llama3	BugsInPy	#Plausible patches	88	Perfect FL, Single-function	Python
AutoSD (2025) [26]	CoT	ChatGPT	Defects4J v1.2, Defects4J v2.0, ARHE	#Correct patches	D4J v1.2: 76, D4J v2.0: 113, ARHE: 187	Perfect FL	Java, Python
CodeCorrector (2025) [32]	CoT	**GPT-4-turbo***, GPT-3.5-turbo	Defects4J v1.2, Defects4J v2.0, QuixBugs	#Correct patches	D4J v1.2: 87, D4J v1.2: 63, QBs: 30 Java, QBs: 38 Py	Single-function	Java, Python
Darwiyanto et al. (2025) [10]	CoT few-shot	**DeepSeek-V3***, GPT-4o, Claude-3.5, Llama3.3, Gemini-1.5-Pro, Grok-2, o1-Preview	QuixBugs	#plausible patches (Avg)	36.6	Single-function	Python
Ehsani et al. (2025) [13]	CoT	Llama 3.3-70B*, GPT-4o-mini	BugsInPy (314 single-function bugs)	Fix rate (%)	79.0	Perfect Fault Localization, Single-function	Python
TypeRepair (2026) [79]	CoT	GPT 4 turbo*, GPT 3.5 turbo	Defects4J v1.2, Defects4J v2.0, QuixBugs	#Correct Patches	D4J v1.2: 82, D4J v1.2: 68, QBs: 37	Single-function	Java, Python
CodeFixer (2024) [33]	ToT	GPT-3.5-Turbo	Defects4J V1.2, QuixBugs	#Correct patches	D4J v1.2: 92, QBs: 40 Java, QBs: 40 Py	Perfect FL, Single-function	Java, Python
DRCodePilot (2024) [82]	Self-Refinement	GPT-4-Turbo	938 issues from Flink and Solr	Full-Match Ratio (%)	Flink: 15.27, Solr: 8.03	-	Java
REx (2024) [57]	Self-Refinement	GPT-4	NLA	Success Rate (%)	73.7	-	C
Sobania et al. (2023) [56]	Conversational	ChatGPT	QuixBugs	#Fixed bugs	31	Single-function	Python
ChatRepair (2024) [68]	Conversational	GPT-3.5-turbo	Defects4J v1.2, Defects4J v2.0, QuixBugs	#Correct patches	D4J v1.2: 114, D4J v2.0: 48, QBs: 40 Java, QBs: 40 Py	Perfect FL	Java, Python
CREF (2024) [72]	Conversational	**GPT-4***, GPT-3.5, Claude-instant-52B, Bard-137B, CodeLlama-13B	TutorCode, TutorCodePlus	AVG-5 (%)	76.6	-	C++
ContrastRepair (2025) [28]	Conversational	GPT-3.5-turbo	Defects4J v1.2, Defects4J v2.0, QuixBugs, HumanEval-Java	#Correct patches	D4J v1.2: 103, D4J v2.0: 40	Perfect FL	Java, Python
InferFix (2023) [24]	Retrieval-Augmented Prompting	Codex	InferredBugs	Top-1 Accuracy (%)	Java: 76.8, C#: 65.6	Perfect FL	Java, C#
RING (2023) [25]	Retrieval-Augmented Prompting	Codex	LaMirage, BIFI, TFix, DeepFix	Pass@k (%)	Excel: 0.82, Python: 0.94, C: 0.63	-	Excel, Power Fx, Python, JavaScript, C, PowerShell.
RLCE (2024) [9]	Retrieval-Augmented Prompting	**GPT-4***, GPT-3.5, PaLM2	RepoBugs	Correct repair (%)	81.45	-	Python
RAGFix (2024) [39]	Retrieval-Augmented Prompting	Llama3-70b	HumanEvalFix	Code Repair Accuracy (%)	78	Single-function	Python
RetypeR (2024) [16]	Retrieval-Augmented Prompting	CodeT5	TypeBugs, BugsInPy	#Correct Patches	TB: 62, BPy: 29	Perfect FL	Python
DSrepair (2025) [46]	Retrieval-Augmented Prompting	**GPT-3.5-turbo***, GPT-4o-mini, DeepSeek-Coder, Codestral	DS-1000	Fix Rate (%)	29.18	-	Python

Continued on next page

Table C.9 – *Continued from previous page*

Approach	Prompting Strategy	LLM backbone	Eval. Bench.	Key Metric	Results	Key Assumptions	Prog. Lang.
ReAPR (2025) [36]	Retrieval-Augmented Prompting	**CodeLlama-7B***, GPT-Neo, Phi-2, DeepSeekCoder- 6.7B	Defects4J 2.0, GitBug-Java	#Plausible Patches, #Correct Patches	D4J: 130 plausible, 68 correct	Perfect FL, Single-function	Java
DeepK (20246 [31]	Retrieval-Augmented Prompting	DeepSeek-v3*, GPT-4o	ACPR, Atcoder	Repair accuracy (%)	ACPR: 90.6, Atcoder: 42.50	Single-function	Python
ExpeRepair (2026) [41]	Retrieval-Augmented Prompting	Claude 4 Sonnet*, Claude 3.5 Sonnet, o1-mini, DeepSeek-R1	SWE-Bench Lite, SWE-Bench Verified	Pass@1 (%)	Lite: 60.3, Verified: 74.6	-	Python
Passerine (2025) [51]	ReAct	Gemini 1.5 Pro	GITS-Eval	Plausible patch rate (%)	73	-	Java, Python, C++, JavaScript
RepairAgent (2025) [4]	ReAct	GPT-3.5	Defects4J v1.2, Defects4J v2.0, GitBug-Java	#Correct patches	D4J v1.2: 74, D4J v2.0: 90, GB-J: 13	Perfect FL	Java

Note: When multiple LLM backbones are listed, the Results column reports the values corresponding to the model annotated with *.

Appendix D. Quality Assurance and Reliability

This appendix presents the complete per-study extraction for approaches classified under Quality Assurance and Reliability strategies, corresponding to the techniques discussed in Section 3.3. These studies prioritize patch correctness and repair robustness through mechanisms such as prompt chaining, self-consistency sampling, prompt tuning, and multi-agent validation pipelines. Each row reports the prompting strategy, LLM backbone, evaluation benchmark(s), primary metric, key results, underlying assumptions, and target programming language(s). When multiple LLM backbones are evaluated, the results column corresponds to the model marked with an asterisk (*).

Table D.10: Summary of studies categorized by quality assurance and reliability

Approach	Prompting Strategy	LLM backbone	Eval. Bench.	Key Metric	Results	Key Assumptions	Prog. Lang.
AutoCodeRover (2024) [81]	Prompt Chaining	GPT-4	SWE-bench, SWE-bench Lite, SWE-bench Devin	Pass@1 (%)	SWB: 12.42, Lite: 19, Devin: 12.63	-	Python
RGD (2024) [23]	Prompt Chaining	**GPT-4o***, GPT-4o-mini	HumanEval, MBPP, APPS-100	Pass@1 (%)	HE: 97.6, MBPP: 83.4, APPS: 63	-	Python
Robin (2024) [3]	Prompt Chaining	GPT-4	Protobuf-net #191, Yaml-DotNet #673	Resolution Success Rate (%)	87.5	-	C#
PATCH (2025) [80]	Prompt Chaining	GPT-3.5-turbo	BFP, Bugs.jar, Bears, Defects4J, QuixBugs	Fix@1 (%), Exact matches (EM), Correct patches (CP)	BFP: 33.97(F1), Bugs: 180 (EM), Bears: 28 (EM), D4J: 169 (CP), QBs: 35 (CP)	Perfect FL	Java
AdverIntent-Agent (2025) [75]	Prompt Chaining	GPT-4o	Defects4J 2.0, HumanEval-Java	Correct patches	D4J: 77, HE-J: 105	Realistic FL	Java
SpecRover (2025) [52]	Prompt Chaining	**Claude 3.5 Sonnet***, GPT-4o	SWE-bench, SWE-bench Lite	Pass@1 efficacy (%)	SWB: 19.31, Lite: 31	-	Python
UniDebugger (2025) [29]	Prompt Chaining	**gpt-4o*** +7 LLMs	Defects4J, QuixBugs, Codeflaws	Correct patches	D4J: 197	-	Java, C, Python
T^3 (2025) [35]	Prompt Chaining	GPT-3.5-turbo*, GPT-4o-mini	MODIT (B2Fs, B2Fm subsets)	Repair Rate (%)	B2Fs: 46.70, B2Fm: 28.20	Perfect FL	Java
Agentless (2025) [65]	Prompt Chaining	GPT 4o*, Claude 3.5 Sonnet	SWE bench (Lite, Lite-S, Verified subsets))	(%) Resolved	Lite: 32, Lite-S: 33.73, Verified: 38.80	-	Python
Abdinabiev et al. (2026) [1]	Prompt Chaining	GPT-4o*, QwenCoder-32B, CodeLlama-34B	Defects4J v1.2, Defects4J v2.0, SWE-Bench Lite	#Correct patches	D4J v1.2: 187, D4J v2.0: 178, SB-L: 87	Perfect FL for main comparison	Java, Python
Cai et al. (2025) [6]	Prompt Tuning	**CodeT5***, GPT-Neo	BugsInPy, Code Refinement, Defects4J, ManySStuBs4J, TFix, xCodeEval	Exact Match (%)	18.03, 2.56, 14.06, 57.29, 25.82, 4.44	-	Java, Python, C, JavaScript
Ahmed et al. (2023) [2]	Self-Consistency	CodeDaVinci-002	MODIT (B2Fs, B2Fm subsets)	Exact Match Accuracy (%)	B2Fs: 31.80, B2Fm: 21.60	Single-function	Java

Note: When multiple LLM backbones are listed, the Results column reports the values corresponding to the model annotated with *.